Implementierung von Counterfactual Regret Minimization in zunehmend komplexen Pokervarianten

vorgelegt von

Friedemann Doll

EDV.Nr.:924315

dem Fachbereich VI – Informatik und Medien
der Berliner Hochschule für Technik Berlin
vorgelegte Bachelorarbeit
zur Erlangung des akademischen Grades

Bachelor of Engineering (B.Eng.)

im Studiengang

Technische Informatik

Tag der Abgabe 21. Januar 2026

Gutachter

Prof. Dr.-Ing. Stefan Edlich	Berliner Hochschule für Technik
Prof. Dr.-Ing. habil. Alexander Löser	Berliner Hochschule für Technik

BHT Berliner Hochschule für Technik

Studiere Zukunft

Inhaltsverzeichnis

1 Einleitung **7**

1.1 Motivation und Relevanz 7

1.2 Zielsetzung und Fragestellungen 8

1.3 Aufbau der Arbeit 9

2 Theoretische Grundlagen **11**

2.1 Grundlagen der Spieltheorie 11

2.1.1 Definition und Konzepte 11

2.1.2 Extensive Games with Imperfect Information 13

2.2 Counter Factual Regret Minimization 15

2.2.1 Definitionen 15

2.2.2 Regret Minimierung 16

2.2.3 Counterfactual Regret 17

2.2.4 Regret Matching 18

2.2.5 CFR Algorithmus 18

2.3 Monte Carlo CFR 19

2.3.1 Allgemeine Definition 19

2.3.2 Chance Sampling CFR 20

2.3.3 External Sampling MCCFR 20

2.3.4 Outcome Sampling MCCFR 20

2.3.5 Theoretische Eigenschaften 21

2.4 CFR+ 22

2.4.1 Unterschiede zu Vanilla CFR 22

2.4.2 Regret Matching+ 22

2.4.3 Tracking Regret 23

2.4.4 Linear Weighted Average 24
2.5 Discounted CFR 25
2.5.1 Motivation 25
2.5.2 Formale Definition 25
2.5.3 Parameterwahl 26
2.5.4 Theoretische Eigenschaften 26
2.6 Exploitability einer Strategie 27
2.6.1 Konventionelle Best Response Berechnung 27
2.6.2 Accelerated Best Response 27
2.6.3 Public State Tree 28
2.6.4 Effiziente Terminal-Node-Evaluierung 28
2.6.5 Game-Spezifische Isomorphismen 28
2.6.6 Parallele Berechnung 29
2.7 Hold'em Games 30
2.7.1 Kuhn Poker 31
2.7.2 Leduc Hold'em 31
2.7.3 Zwölf Karten Poker 32
2.7.4 Rhode Island Hold'em 32
2.7.5 Limit Texas Hold'em 32

3 Analyse 35
3.1 Spielvarianten 35
3.2 CFR Solver 35
3.3 Evaluationsmethoden 36

4 Architektur und Design 37
4.1 Game Environment 38
4.2 CFR Solver 38
4.2.1 Dynamischer Spielbaum 38
4.2.2 Statischer Spielbaum 43
4.3 Evaluationsmethoden 46
4.3.1 Accelerated Best Response 46

5 Implementierung **49**

5.1 Entwicklungsvorgehen 49

5.1.1 Iteratives Vorgehen statt „Big Design Up Front“ 49

5.1.2 Phase 1: Grundgerüst und erste Solver 49

5.1.3 Phase 2: Evaluation als Diagnosetool (Public State Tree und Best Response) 50

5.1.4 Phase 3: Das zentrale Architekturproblem mit Chance Nodes . . 50

5.1.5 Phase 4: Späte Fehlererkennung und Konsequenzen 50

5.1.6 Phase 5: Wechsel zur Tree-basierten Implementierung 50

5.1.7 Phase 6: Pragmatische Entscheidungen und Erweiterungen . . . 51

5.1.8 Einordnung für die folgenden Abschnitte 51

5.2 Probleme und Herausforderungen 51

5.2.1 Fehlende explizite Chance Nodes für öffentliche Karten 51

5.2.2 Konsequenzen für Public State Tree und Best Response 52

5.2.3 Sampling-Solver: Ziel nicht erreicht 52

5.2.4 Skalierungsgrenzen am Beispiel großer Hold’em-Varianten . . . 52

5.2.5 Nicht umgesetzte Abstraktion: Hand-strength-squared 53

5.3 Erkenntnisse und Reflexion 53

5.3.1 Was sich im Projektverlauf als wichtig erwiesen hat 53

5.3.2 Lessons Learned aus der Implementierung 53

5.3.3 Was ich beim nächsten Mal anders machen würde 53

6 Evaluation **55**

6.1 Ergebnisse 55

6.2 Vergleich der Algorithmen 55

6.3 Vergleich mit Literaturwerten und anderen Modellen 55

7 Fazit **57**

7.1 Zielerreichung und Einordnung 57

7.1.1 Kurzüberblick 57

7.1.2 Zielerreichung (Bezug zu Kapitel 1) 57

7.1.3 Rückbindung an Hypothesen und Fragestellungen 58

7.2 Limitierungen 58

7.2.1 Iterationslaufzeit als primäre Skalierungsgrenze 58
7.2.2 Speicherbedarf bei Tree-basierten Varianten 59
7.2.3 Evaluation hängt von PST und Best Response ab 59
7.2.4 Messdichte und Zeitmessungen 59
7.3 Future Work 59
7.3.1 Implementierung und Engineering 59
7.3.2 Algorithmische und wissenschaftliche Erweiterungen 60

8 Anhang **61**
8.1 M-Wert 61

Literaturverzeichnis **63**

Kapitel 1

Einleitung

1.1 Motivation und Relevanz

Im Alltag werden ständig Entscheidungen getroffen, ohne dass Entscheidungsträger vollständig über alle relevanten Umstände informiert sind. Um dennoch handlungsfähig zu bleiben, treffen Menschen bewusst oder unbewusst Annahmen über unbekannte Aspekte einer Situation. Das Verhalten von Akteuren in solchen strategischen Entscheidungssituationen lässt sich durch spieltheoretische Modelle beschreiben. Die Spieltheorie umfasst sowohl Modelle für Situationen bzw. Spiele mit vollständiger als auch mit unvollständiger Information. Ein prominentes Beispiel für ein Spiel mit unvollständiger Information ist Poker, insbesondere Hold'em-Varianten wie Texas Hold'em oder Omaha Hold'em.

Ich habe im letzten Jahr ein ausgeprägtes Interesse an dem Spiel No-Limit Texas Hold'em entwickelt. Wenn man sich mit Strategien für dieses Spiel auseinandersetzt, stößt man schnell auf ein Konzept, das „Game Theory Optimized" (GTO) genannt wird. Dieses Konzept hat innerhalb der letzten Jahre immer mehr an Aufmerksamkeit und Relevanz gewonnen und ist heutzutage kaum noch aus der Pokerwelt wegzudenken.
Eine GTO-Strategie ist im Grunde eine spieltheoretisch optimale Lösung, welche durch den Einsatz von Algorithmen approximiert wird. Diese Strategien sind für Spiele mit einem enorm großen Zustandsraum, wie No Limit Hold'em, so komplex, dass sie von Menschen nicht vollständig erlernt und konsistent angewendet werden können. Pokerprofis sind allerdings in der Lage, diese Strategien in abstrahierter Form zu adaptieren, indem sie grundlegende strategische Prinzipien und Verhaltensmuster erkennen und anschließend auf ihre Spielsituationen anwenden. Die intensive Auseinandersetzung mit GTO-Strategien in meiner Freizeit weckte mein Interesse an den algorithmischen Grundlagen, die zur Berechnung solcher Strategien verwendet werden.

Ein zentraler Algorithmus welcher verwendet wird um solche Strategien zu finden ist Counterfactual Regret Minimization (CFR).
Seit seiner Einführung durch Zinkevich et al. (2007) [Zinkevich et al., 2007] hat sich CFR als Standardansatz etabliert und bildet die Grundlage für die erfolgreichsten Poker-KIs.
Zu den größten Erfolgen gehören dabei:
1. Cepheus [Bowling et al., 2015, Tammelin et al., 2015]
Mit Cepheus wurde das Spiel Heads-Up Limit Texas Hold'em erstmals gelöst.

2. Libratus [Brown and Sandholm, 2018, Brown and Sandholm, 2017]
Libratus erweiterte diesen Erfolg dann auf die No-Limit Variante.
3. Pluribus [Brown and Sandholm, 2019b].
Durch Pluribus wurde gezeigt, dass CFR basierte Ansätze auch in Mehrspieler Szenarien erfolgreich sein können.

1.2 Zielsetzung und Fragestellungen

Im Rahmen dieser Arbeit wird Counterfactual Regret Minimization in Python implementiert und auf verschiedenen Pokervarianten unterschiedlicher Komplexität angewendet. Dabei wird untersucht, wie sich die Leistungsfähigkeit des Algorithmus bei steigender Spielkomplexität verhält. Wichtige Parameter die betrachtet werden um die Leistungsfähigkeit einzuschätzen sind unter anderem die Laufzeit, der Speicherverbrauch und die Qualität der trainierten Strategie. Durch die Anwendung von CFR auf unterschiedlich große Pokervarianten soll ermittelt werden, ab welchem Komplexitätsgrad algorithmische und implementierungstechnische Optimierungen vorgenommen werden müssen.

In meine Exposé waren drei Spiele geplant, Kuhn Poker als simpelste existierende Pokervariante, Leduc Hold'em als etabliertes Benchmark Spiel und eine selbst entwickelte Variante mit 12 Karten als nächste Stufe in der Komplexitätsleiter.
Zusätzlich sollten optimierte CFR Verfahren wie CFR+ und Monte Carlo CFR implementiert werden und deren Auswirkung auf die Konvergenzgeschwindigkeit untersucht Um die Qualität einer trainierten Strategie zu bewerten wird berechnet wie Ausnutzbar diese ist. Dies geschieht mithilfe eines Best Response Agents, welcher immmer die best mögliche Aktion gegen eine Strategie spielt. Im Verlauf des Projekts wurden diese Ziele grundsätzlich beibehalten, jedoch wurde Monte Carlo CFR durch Discounted CFR ersetzt.

Im Folgenden formuliere ich drei Forschungsfragen, die im Verlauf des Projekts beantwortet werden sollen.

1. **Algorithmusvergleich:** Wie groß sind die Unterschiede im Konvergenverhalten der verschiedenen CFR Varianten?

2. **Implementierungsoptimierungen:** Welche Implementierungsoptimierungen sind erforderlich, damit das Training bei wachsender Spielgröße praktikabel bleibt?

3. **Limitationen:** Durch welche Komponenten wird bei steigender Spielgröße die geplante Implementierung limitiert?

1.3 Aufbau der Arbeit

Die Arbeit gliedert sich in sieben Kapitel, die den Weg von den theoretischen Grundlagen über die Architektur und Implementierung bis hin zur Evaluation der trainierten Strategien beschreiben.

- **Kapitel 2** stellt die theoretischen Grundlagen vor.
  Zu Beginn werden grundlegende spieltheoretische Begriff eingefürt und eine formale Definition eines „Extensive Game with Imperfect Information“ gegeben. Anschließend wird die Theorie des CFR Algorithmus detailiert erkläutert, gefolgt von den optimierten Varianten
  Anschließend wird die Theorie des Accelerated Best Response Agents erläutert, der zur Evaluation der Strategiequalität verwendet wird. Abschließend werden mehrere Poker- und Hold'em-Varianten vorgestellt und hinsichtlich ihrer Komplexität eingeordnet, darunter auch Varianten, die im Rahmen dieser Arbeit nicht vollständig gelöst werden.

- **Kapitel 3** analysiert die Anforderungen an die Implementierung. Es wird ausserdem die Auswahl der betrachteten Pokervarianten und CFR-Varianten, sowie Evaluationsmethoden begründet.

- **Kapitel 4** beschreibt die Systemarchitektur anhand von UML-Diagrammen, wobei sowohl die Struktur der Spielumgebung als auch die des CFR-Solvers erläutert werden. Die Designentscheidungen und die grundlegenden Abläufe werden dargestellt.

- **Kapitel 5** liefert einen Überblick über den Entwicklungsverlauf und hebt dabei hervor was funktioniert hat und was nicht.

- **Kapitel 6** veranschaulicht den Konvergenzverlauf der trainierten Strategien und evaluiert die Ergebnisse. Diese werden wenn möglich mit Literaturwerten verglichen um die Richtigkeit zu verifizieren

- **Kapitel 7** fasst die Erkenntnisse zusammen, diskutiert Limitationen und gibt einen Ausblick auf mögliche Erweiterungen.

Kapitel 2

Theoretische Grundlagen

2.1 Grundlagen der Spieltheorie

In diesem Kapitel wird ein kurzer Überblick darüber gegeben, was Spieltheorie ist und wo sie angewendet wird. Dabei wird definiert, was im spieltheoretischen Kontext unter einem *Spiel* und einer *Lösung* verstanden wird. Anschließend werden die wichtigsten Typen von Spielen umrissen und der Typ extensiver Spiele mit imperfekter Information näher betrachtet. Als Grundlage für dieses Kapitel dient das Buch „A Course in Game Theory“ [Osborne and Rubinstein, 1994].

2.1.1 Definition und Konzepte

Die Spieltheorie beschäftigt sich mit der Analyse strategischer Interaktionen zwischen Entscheidungsträgern und bietet hierfür eine Vielzahl analytischer Methoden. Diese basieren auf zwei zentralen Annahmen [Osborne and Rubinstein, 1994, Kap. 1.1].

Die erste Annahme ist, dass Entscheidungsträger rational handeln. Ein rationaler Entscheidungsträger kennt seine Handlungsmöglichkeiten, kann Erwartungen über unbekannte Situationen bilden und hat klare Vorstellungen darüber, welche Ergebnisse er bevorzugt. Er wählt seine Aktion bewusst nach einem Prozess der Optimierung. Liegt keine Unsicherheit vor, wählt er die Aktion, die zu den für ihn besten Konsequenzen führt. Diese Präferenzen lassen sich durch eine Nutzenfunktion beschreiben, die jedem möglichen Ergebnis einen Wert zuordnet. Die beste Aktion ist die, die den höchsten Nutzen erzielt. Sind die Konsequenzen von Aktionen unsicher, maximiert ein rationaler Entscheidungsträger den erwarteten Nutzen. Hierfür muss er in der Lage sein, Wahrscheinlichkeiten für unsichere Ereignisse abzuschätzen [Osborne and Rubinstein, 1994, Kap. 1.4].

Die zweite Annahme ist, dass Entscheidungsträger ihr Wissen oder ihre Erwartungen über das Verhalten anderer Spieler bei ihrer Entscheidungsfindung berücksichtigen. Dies nennt man auch strategisches Handeln. Spieltheoretisches Denken berücksichtigt, dass jeder Entscheidungsträger vor seiner Entscheidung versucht, Informationen über das Verhalten der anderen Spieler zu erhalten. Im Gegensatz dazu nimmt die Theorie des Wettbewerbsgleichgewichts an, dass jeder Akteur sich nur für Umgebungsparameter

wie Preise interessiert, auch wenn diese durch die Aktionen aller Akteure bestimmt werden [Osborne and Rubinstein, 1994, Kap. 1.3].
Ein anschauliches Beispiel hierfür ist das Gefangenendilemma. Zwei Verdächtige werden getrennt verhört und müssen jeweils entscheiden, ob sie schweigen oder den anderen belasten. Schweigen beide, erhalten beide zwei Jahre Haft. Wenn beide gestehen, erhalten sie fünf Jahre Haft. Gesteht jedoch nur einer, wird dieser freigelassen, während der andere zu zehn Jahren Haft verurteilt wird. Die optimale Entscheidung hängt davon ab was für Tendenzen der andere Entscheidungsträger hat. Strategisches Handeln bedeutet, solche Erwartungen über das Verhalten anderer Spieler in die Entscheidungsfindung mit einzubeziehen [Osborne and Rubinstein, 1994, Kap. 1.1].

Spieltheoretische Modelle befinden sich auf einer hohen Abstraktionsebene und können daher verwendet werden, um ein breites Spektrum an Phänomenen zu betrachten. Zur formalen Darstellung dieser Modelle werden mathematische Definitionen genutzt. Die zugrunde liegenden Konzepte sind jedoch nicht primär mathematischer Natur und lassen sich auch ohne formale Notation erläutern.
Die Spieltheorie findet Anwendung in unzähligen Bereichen. Zu den klassischen Anwendungsgebieten zählen die Analyse wirtschaftlicher und politischer Wettbewerbssituationen. Darüber hinaus wird sie auch in der Evolutionsbiologie, der Soziologie und anderen Disziplinen genutzt, um strategische Interaktionen zu modellieren.
[Osborne and Rubinstein, 1994, Kap. 1.1].

Ein Spiel beschreibt eine strategische Interaktion zwischen Spielern mit festgelegten Rahmenbedingungen. Es definiert, welche Aktionen den Spielern zur Verfügung stehen und welche Ergebnisse in ihrem Interesse sind. Das Spiel legt damit die Struktur und die Regeln der Interaktion fest, bestimmt aber nicht, wie sich die Spieler tatsächlich verhalten werden.
Die Frage, welche Aktionen die Spieler tatsächlich ausführen, wird nicht durch das Spiel selbst beantwortet, sondern durch die Lösung des Spiels. Eine Lösung ist ein Konzept, das systematisch beschreibt, welche Ergebnisse in einer Klasse von Spielen unter bestimmten Verhaltensannahmen zu erwarten sind. Sie liefert damit eine Vorhersage über das tatsächliche Verhalten der Spieler unter den getroffenen Annahmen über Rationalität und strategisches Denken [Osborne and Rubinstein, 1994, Kap. 1.2].
Spiele können anhand verschiedener Kriterien systematisch kategorisiert werden. Grundlegend für alle spieltheoretischen Modelle ist das Konzept des Spielers. Ein Spieler kann dabei sowohl eine einzelne Person als auch eine Gruppe von Personen darstellen, die gemeinsam eine Entscheidung treffen.

Die erste grundlegende Unterscheidung betrifft kooperative und nicht kooperative Spiele. Nicht kooperative Spiele modellieren die Aktionen einzelner Spieler. Kooperative Spiele hingegen konzentrieren sich darauf auf Gruppen von Spielern. Dabei wird nicht betrachtet, wie diese Gruppen intern funktionieren [Osborne and Rubinstein, 1994, Kap. 1.2].

Eine weitere wichtige Kategorisierung betrifft die Darstellungsform des Spiels. Hierbei werden strategische Spiele, auch als Spiele in Normalform bezeichnet, von extensiven Spielen unterschieden. Bei strategischen Spielen wählen die Spieler ihre Strategie zu Beginn des Spiels und passen diese während des Spielverlaufs nicht mehr an. Die Entscheidungen werden zudem simultan getroffen, sodass keine Informationen über die Aktionen der Gegenspieler verfügbar sind. Extensive Spiele hingegen spezifizieren die möglichen Reihenfolgen von Ereignissen und erlauben es jedem Spieler, seinen Akti-

onsplan nicht nur zu Beginn, sondern auch bei jeder weiteren Entscheidung zu überdenken [Osborne and Rubinstein, 1994, Kap. 1.2].

Die letzte hier zu nennende Unterscheidung betrifft den Informationsstand der Spieler. Bei Spielen mit perfekter Information sind alle Teilnehmer vollständig über die Züge der anderen Spieler informiert. Bei Spielen mit imperfekter Information hingegen können die Spieler unvollständig über die Aktionen der anderen oder über Ausgänge von Zufallsereignissen informiert sein [Osborne and Rubinstein, 1994, Kap. 1.2].

2.1.2 Extensive Games with Imperfect Information

Nachdem die grundlegenden Konzepte der Spieltheorie eingeführt wurden, widmet sich dieses Kapitel den extensiven Spielen mit imperfekter Information. Diese Spielklasse ist für die vorliegende Arbeit von zentraler Bedeutung, da die meisten Pokerspiele, insbesondere Hold'em-Varianten, zu dieser Kategorie gehören.

Definition

Ein extensives Spiel mit imperfekter Information modelliert eine sequenzielle Interaktion zwischen Spielern, bei der nicht alle Spieler stets vollständig über alle vergangenen Aktionen informiert sind. Dabei werden die möglichen Spielverläufe als Historien erfasst, wobei eine Historie eine Abfolge von Aktionen darstellt. Zu jedem Zeitpunkt ist entweder ein Spieler oder der Zufall am Zug.
Um die imperfekte Information darzustellen, werden Informationssets verwendet. Diese fassen Zustände zusammen, die ein Spieler nicht voneinander unterscheiden kann. Dies kann sowohl vergangene Aktionen anderer Spieler als auch Ausgänge von Zufallsereignissen betreffen.

Ein extensives Spiel mit imperfekter Information lässt sich formal durch folgende Komponenten definieren:

- Eine Menge von Historien H.q Eine Historie ist eine endliche oder unendliche Sequenz von Aktionen, welche entweder von Spielern oder dem Zufall getätigt werden.
  Die Menge H erfüllt folgende Eigenschaften:
  - Die leere Sequenz $\emptyset$ ist ein Element von H.
  - Ist $(a^k)_{k=1,\dots,K} \in H$ (wobei K unendlich sein kann) und $L < K$, dann gilt $(a^k)_{k=1,\dots,L} \in H$. Das bedeutet, wenn eine Sequenz ein Teil von H ist, dann ist auch jeder Präfix dieser Sequenz ein Teil von H.
  - Wenn für eine unendliche Folge $(a^k)_{k=1}^{\infty}$ die Bedingung $(a^k)_{k=1,\dots,L} \in H$ für jede positive ganze Zahl L erfüllt ist, dann gilt $(a^k)_{k=1}^{\infty} \in H$.
- Eine Historie $(a^k)_{k=1,\dots,K} \in H$ ist terminal, wenn sie unendlich ist oder wenn kein a^{K+1} existiert, so dass $(a^k)_{k=1,\dots,K+1} \in H$ gilt. Man bezeichnet die Menge aller terminalen Historien mit Z.
  Die Menge der Aktionen, die nach einer nicht terminalen Historie h verfügbar sind, wird mit $A(h) = \{a : (h, a) \in H\}$ bezeichnet.

- Die Funktion P ordnet jeder nicht terminalen Historie $h \in H \setminus Z$ einen Spieler aus N oder c zu. $P(h)$ bezeichnet den Spieler, der nach h am Zug ist. Wenn $P(h) = c$ ist, bestimmt der Zufall die nächste Aktion.

- Die Funktion f_c ordnet jeder Historie h mit $P(h) = c$ eine Zufallsverteilung $f_c(\cdot \mid h)$ auf $A(h)$ zu. $f_c(a \mid h)$ ist die Wahrscheinlichkeit, dass nach h die Aktion a folgt. Diese Verteilungen sind unabhängig voneinander.

- Für jeden Spieler $i \in N$ wird die Menge der Historien h mit $P(h) = i$ in Informationssets unterteilt. Diese Informationspartition wird bezeichnet mit$\mathcal{I}_i$. Ein Element $I_i \in \mathcal{I}_i$ heißt Informationsset von Spieler i. Historien h und h', die zum selben Informationsset I_i gehören, sind für Spieler i nicht unterscheidbar. Wesentlich ist dabei, dass alle Historien in einem Informationsset I_i die gleichen verfügbaren Aktionen haben, also $A(h) = A(h')$ für alle $h, h' \in I_i$. Für ein Informationsset I_i bezeichnet $A(I_i)$ die verfügbaren Aktionen und $P(I_i)$ den Spieler, der am Zug ist.

- Für jeden Spieler $i \in N$ ist eine Präferenzrelation $\succeq_i$ über Lotterien auf Z gegeben. Eine Lotterie ist hierbei eine Wahrscheinlichkeitsverteilung über terminale Historien. Diese Präferenzen lassen sich durch eine Nutzenfunktion $u_i : Z \to \mathbb{R}$ repräsentieren, sodass für zwei Lotterien L_1, L_2 gilt: $L_1 \succeq_i L_2 \iff \mathbb{E}_{z \sim L_1}[u_i(z)] \geq \mathbb{E}_{z \sim L_2}[u_i(z)]$.

Formale Definition entspricht [Osborne and Rubinstein, 1994, Ch. 11, Sec. 11.1, Def. 200.1].

Strategien

In extensiven Spielen mit imperfekter Information gibt es verschiedene Möglichkeiten, wie die Strategien von Spielern definiert sein können.
Eine reine Strategie von Spieler $i \in N$ ordnet jedem Informationsset $I \in \mathcal{I}_i$ genau eine zulässige Aktion $s_i(I) \in A(I)$ zu. Sie legt somit einen vollständigen Handlungsplan für alle Informationslagen von i fest.
Eine gemischte Strategie α_i ist eine Wahrscheinlichkeitsverteilung über der Menge der reinen Strategien von i.
Eine behavioral Strategie β_i hingegen ist eine Sammlung unabhängiger Wahrscheinlichkeitsverteilungen $(\beta_i(I))_{I \in \mathcal{I}_i}$, wobei $\beta_i(I)$ eine Wahrscheinlichkeitsverteilung über die Aktionen $A(I)$ am Informationsset I ist [Osborne and Rubinstein, 1994, Ch. 11, Sec. 11.4, Def. 212.1].
Der Unterschied liegt darin, dass bei einer gemischten Strategie vor Spielbeginn zufällig eine komplette reine Strategie ausgewählt wird, die dann während des gesamten Spiels deterministisch befolgt wird.
Im Gegensatz dazu wird bei einer behavioral Strategie in jedem Informationsset unabhängig eine Wahrscheinlichkeitsverteilung über die verfügbaren Aktionen verwendet, sodass die zufällige Auswahl der Aktionen während des Spiels erfolgt [Osborne and Rubinstein, 1994, Ch. 11, Sec. 11.4].
Ein extensives Spiel hat perfect recall, wenn jeder Spieler zu jedem Zeitpunkt alles erinnert, was er in der Vergangenheit wusste und welche Aktionen er selbst getroffen hat [Osborne and Rubinstein, 1994, Ch. 11, Sec. 11.1].
Unter der Annahme von perfect recall erzeugen beide Repräsentationen outcome-

äquivalente Wahrscheinlichkeitsverteilungen über die terminalen Historien [Osborne and Rubinstein, 1994, Ch. 11, Sec. 11.4, Prop. 214.1].

Ein Profil von behavioral Strategien $\beta = (\beta_i)_{i \in N}$ zusammen mit den Zufallszügen f_c bestimmt eine Wahrscheinlichkeitsverteilung über die terminalen Historien Z. Daraus lässt sich für jede Auszahlungsfunktion u_i der Erwartungswert $\mathbb{E}_{z \sim O(\beta, f_c)}[u_i(z)]$ berechnen.

2.2 Counter Factual Regret Minimization

Dieses Kapitel stellt die theoretischen Grundlagen von *Counterfactual Regret Minimization* (CFR), dem zentralem Algorithmus dieser Arbeit vor. Als fachliche Grundlage dient das Paper „Regret Minimization in Games with Incomplete Information“ [Zinkevich et al., 2007], in dem CFR erstmals vorgestellt wurde.
Zunächst wird das Konzept des Regrets eingeführt. Anschließend wird definiert, wann ein Algorithmus als regret minimierend gilt. Darauf aufbauend wird das von Zinkevich et al. eingeführte Konzept des Counterfactual Regret sowie Regret Matching erläutert. Abschließend fügen sich die einzelnen Komponenten zu einem Algorithmus zusammen.

2.2.1 Definitionen

Dieser Abschnitt führt die grundlegenden Definitionen und Konzepte ein, die für das Verständnis von Counterfactual Regret Minimization erforderlich sind.

Wahrscheinlichkeiten und Werte

Für ein Strategieprofil σ werden folgende Wahrscheinlichkeiten und Werte definiert:

Die Wahrscheinlichkeit, dass eine Historie h auftritt, wenn alle Spieler gemäß σ handeln, wird mit $\pi^\sigma(h)$ bezeichnet. Diese Wahrscheinlichkeit kann in ein Produkt zerlegt werden:

$$\pi^\sigma(h) = \prod_{i \in N \cup \{c\}} \pi_i^\sigma(h), \tag{2.1}$$

wobei $\pi_i^\sigma(h)$ die Wahrscheinlichkeit bezeichnet, dass Spieler i in all seinen Entscheidungsknoten auf dem Pfad zu h gemäß σ die Aktionen gewählt hat, die zu h führen. Das Produkt der Wahrscheinlichkeiten aller anderen Spieler (inklusive Zufallsereignisse) wird mit $\pi_{-i}^\sigma(h)$ bezeichnet.

Für ein Informationsset I wird die Erreichbarkeitswahrscheinlichkeit definiert als:

$$\pi^\sigma(I) = \sum_{h \in I} \pi^\sigma(h). \tag{2.2}$$

Analog sind $\pi_i^\sigma(I)$ und $\pi_{-i}^\sigma(I)$ als die Wahrscheinlichkeiten definiert, dass Spieler i beziehungsweise die anderen Spieler das Informationsset I erreichen.

Der erwartete Nutzen für Spieler i unter Strategieprofil σ, auch *Overall Value* genannt, wird wie folgt berechnet:

$$u_i(\sigma) = \sum_{h \in Z} u_i(h)\pi^{\sigma}(h), \tag{2.3}$$

Nash-Gleichgewicht

Ein Nash-Gleichgewicht beschreibt ein Strategienprofil, bei dem kein Spieler durch eine einseitige Änderung seiner Strategie seinen erwarteten Nutzen erhöhen kann [Osborne and Rubinstein, 19 Ch. 11, Sec. 11.5]. Ein Nash-Gleichgewicht für ein Zweispieler Nullsummenspiel ist ein Strategieprofil $\sigma = (\sigma_1, \sigma_2)$, dass folgende Ungleichungen erfüllt:

$$u_1(\sigma) \geq \max_{\sigma_1' \in \Sigma_1} u_1(\sigma_1', \sigma_2), \qquad u_2(\sigma) \geq \max_{\sigma_2' \in \Sigma_2} u_2(\sigma_1, \sigma_2') \tag{2.4}$$

([Zinkevich et al., 2007, Eq. 1]).

Eine Approximation eines Nash-Gleichgewichts, auch ε-Nash-Gleichgewicht genannt, ist ein Strategieprofil σ, dass folgende Ungleichungen erfüllt:

$$u_1(\sigma) + \varepsilon \geq \max_{\sigma_1' \in \Sigma_1} u_1(\sigma_1', \sigma_2), \qquad u_2(\sigma) + \varepsilon \geq \max_{\sigma_2' \in \Sigma_2} u_2(\sigma_1, \sigma_2') \tag{2.5}$$

([Zinkevich et al., 2007, Eq. 2]).

2.2.2 Regret Minimierung

Regret misst den Unterschied zwischen dem maximal erreichbaren Nutzen unter Verwendung der zu diesem Zeitpunkt bestmöglichen Strategie und dem tatsächlich erzielten Nutzen.
Um Regret Minimierung zu definieren, wird das wiederholte Spielen eines extensiven Spiels betrachtet. Sei σ_i^t die von Spieler i in Runde t verwendete Strategie. Der durchschnittliche *Overall Regret* von Spieler i zum Zeitpunkt T ist:

$$R_i^T = \frac{1}{T} \max_{\sigma_i^* \in \Sigma_i} \sum_{t=1}^{T} \big(u_i(\sigma_i^*, \sigma_{-i}^t) - u_i(\sigma^t)\big). \tag{2.6}$$

([Zinkevich et al., 2007, Eq. 3])

Die Durchschnittsstrategie $\bar{\sigma}_i^T$ für Spieler i von Zeit 1 bis T wird für jedes Informationsset $I \in \mathcal{I}_i$ und jede Aktion $a \in A(I)$ definiert als:

$$\bar{\sigma}_i^T(I)(a) = \frac{\sum_{t=1}^{T} \pi_i^{\sigma^t}(I)\, \sigma_i^t(I)(a)}{\sum_{t=1}^{T} \pi_i^{\sigma^t}(I)}. \tag{2.7}$$

([Zinkevich et al., 2007, Eq. 4])

Die Durchschnittsstrategie gewichtet Aktionen proportional zur Erreichbarkeitswahrscheinlichkeit $\pi_i^{\sigma^t}(I)$ des Informationssets I in Runde t.

Es besteht ein wichtiger Zusammenhang zwischen Regret und Nash-Gleichgewichten. In einem Nullsummenspiel gilt: Ist der durchschnittliche Overall Regret beider Spieler zum Zeitpunkt T kleiner als ε, dann ist $\bar{\sigma}^T$ ein 2ε-Nash-Gleichgewicht ([Zinkevich et al., 2007, Thm. 2]).

Ein Algorithmus zur Bestimmung von σ_i^t ist Regret-minimierend, wenn der durchschnittliche Overall Regret von Spieler i unabhängig von der Strategiefolge σ_{-i}^t des Gegners gegen Null konvergiert, während t gegen Unendlich läuft. Daher können Regret-minimierende Algorithmen im Self-Play verwendet werden, um ein approximatives Nash-Gleichgewicht zu berechnen.

2.2.3 Counterfactual Regret

Die zentrale Idee welche Zinkevich et. al. in ihrem Paper "Regret Minimization in Games with Incomplete Information"präsentieren ist, den gesamten Regret Wert in eine Menge aus additiven Termen zu zerlegen. Diese Terme können dann unabhängig voneinander minimiert werden. Insbesondere wird ein völlig neues Regret Konzept mit dem Namen *Counterfactual Regret* eingeführt, welches für jedes Informationsset definiert ist.

Um das Konzept zu erläutern, wird ein Informationsset $I \in \mathcal{I}_i$ betrachtet.
Die *Counterfactual Utility* $u_i(\sigma, I)$ ist der erwartete Nutzen unter der Bedingung, dass I erreicht wird und alle Spieler mit σ agieren, während Spieler i so agiert um den Zustand I zu erreichen. Mit $\pi^\sigma(h, h')$ als Wahrscheinlichkeit, von h nach h' zu gelangen, gilt

$$u_i(\sigma, I) = \frac{\sum_{h \in I,\, h' \in Z} \pi_{-i}^\sigma(h)\, \pi^\sigma(h, h')\, u_i(h')}{\pi_{-i}^\sigma(I)}. \tag{2.8}$$

([Zinkevich et al., 2007, Eq. 5])

Für alle $a \in A(I)$ ist $\sigma|_{I \to a}$ ein Strategieprofil, das identisch zu σ ist, außer dass Spieler i immer die Aktion a ausführt, wenn er das Informationsset I erreicht. Der *Immediate Counterfactual Regret* ist

$$R_{i,\mathrm{imm}}^T(I) = \frac{1}{T} \sum_{t=1}^{T} \max_{a \in A(I)} \pi_{-i}^{\sigma^t}(I) \left(u_i(\sigma^t|_{I \to a}, I) - u_i(\sigma^t, I) \right). \tag{2.9}$$

([Zinkevich et al., 2007, Eq. 6])

Dieser Term misst den Regret im Informationsset I, gewichtet mit der *Counterfactual Probability*, dass I erreicht werden würde, wenn Spieler i es erzwingen wollte. Da hauptsächlich die positiven Regretwerte relevant sind, sind die positiven Immediate Counterfactual Regret Werte definiert als

$$R_{i,\mathrm{imm}}^{T,+}(I) = \max\left(R_{i,\mathrm{imm}}^T(I), 0\right). \tag{2.10}$$

Theorem 3 beschreibt die erste große Erkenntnis des Papers.

$$R_i^T \leq \sum_{I \in \mathcal{I}_i} R_{i,\mathrm{imm}}^{T,+}(I) \tag{2.11}$$

([Zinkevich et al., 2007, Thm. 3])
Das heißt, wenn der Immediate Counterfactual Regret minimiert wird, dann wird auch der Average Overall Regret minimiert.

2.2.4 Regret Matching

Mit folgender Formel wird der Counterfactual Regret für eine konkrete Aktion a in einem Informationsset I berechnet.

$$R_i^T(I,a) = \frac{1}{T}\sum_{t=1}^{T} \pi_{-i}^{\sigma^t}(I)\left(u_i(\sigma^t|_{I\to a}, I) - u_i(\sigma^t, I)\right) \tag{2.12}$$

([Zinkevich et al., 2007, Eq. 7])

Dabei sei $R_i^{T,+}(I,a) = \max(R_i^T(I,a), 0)$ als positiver Anteil des Immediate Counterfactual Regret für Aktion a definiert.

Die Strategie für Iteration $T+1$ wird dann durch Regret Matching bestimmt:

$$\sigma_i^{T+1}(I)(a) = \begin{cases} \frac{R_i^{T,+}(I,a)}{\sum_{a'\in A(I)} R_i^{T,+}(I,a')} & \text{falls } \sum_{a'\in A(I)} R_i^{T,+}(I,a') > 0 \\ \frac{1}{|A(I)|} & \text{in jedem anderen Fall.} \end{cases} \tag{2.13}$$

([Zinkevich et al., 2007, Eq. 8])

Diese Formel weist Aktionen Wahrscheinlichkeiten proportional zu ihrem positiven Regret aus dem Nichtwählen dieser Aktion zu. Gibt es keine positiven Regrets, werden alle Aktionen gleichverteilt gewählt.

Theorem 4 zeigt, dass bei Verwendung von Gleichung 8 der Immediate Counterfactual Regret und der Overall Regret folgendermaßen beschränkt sind:

$$R_{i,\text{imm}}^T(I) \leq \frac{\Delta_{u,i}\sqrt{|A_i|}}{\sqrt{T}}, \tag{2.14}$$

$$R_i^T \leq \frac{\Delta_{u,i}|\mathcal{I}_i|\sqrt{|A_i|}}{\sqrt{T}}, \tag{2.15}$$

wobei $\Delta_{u,i} = \max_z u_i(z) - \min_z u_i(z)$ der Nutzenbereich und $|A_i| = \max_{h:P(h)=i} |A(h)|$ die maximale Anzahl verfügbarer Aktionen für Spieler i ist ([Zinkevich et al., 2007, Thm. 4]). Die Schranke für den Average Overall Regret ist linear in der Anzahl der Informationssets $|\mathcal{I}_i|$ und fällt asymptotisch mit $O(1/\sqrt{T})$. Zusammen mit Theorem 2 folgt, dass die Durchschnittsstrategie $\bar{\sigma}^T$ im Selbstspiel gegen ein Nash-Gleichgewicht konvergiert ([Zinkevich et al., 2007, Thm. 2]).

2.2.5 CFR Algorithmus

Das CFR-Verfahren verbindet die Komponenten aus den vorherigen Abschnitten zu einer wiederholten Selbstspielprozedur. In jeder Iteration werden die Strategien gemäß Gleichung 8 aktualisiert. Für jedes Informationsset I und jede Aktion a werden die kumulierten Regretwerte $R_i^t(I,a)$ gespeichert und nach jeder Iteration aktualisiert. Parallel dazu wird die Durchschnittsstrategie gemäß Gleichung 4 aktualisiert. Nach T Iterationen gibt der Algorithmus $(\bar{\sigma}_1^T, \bar{\sigma}_2^T)$ als approximatives Nash-Gleichgewicht zurück.

2.3 Monte Carlo CFR

Der CFR Algorithmus benötigt in jeder Iteration eine vollständige Traversierung des gesamten Spielbaums. Monte Carlo CFR ist ein im Jahr 2009 von Lanctot et al. [Lanctot et al., 2009] veröffentlichtes Framework von Algorithmen, welche diesen Aufwand durch verschiedene Sampling-Varianten reduzieren. In diesem Kapitel wird zuerst die Grundidee des Frameworks erklärt und anschließend genauer auf die einzelnen Sampling Verfahren eingegangen. Abschließend wird ein neues Maß zur Abschätzung der Spielkomplexität eingeführt und die theoretischen Eigenschaften der Algorithmen besprochen.

2.3.1 Allgemeine Definition

MCCFR fasst die Menge aller terminalen Historien Z in Teilmengen zusammen, die als Blöcke bezeichnet werden. Die Menge dieser Teilmengen ist definiert als $Q = \{Q_1, \ldots, Q_r\}$. In jeder Iteration wird zufällig ein Block ausgewählt und nur die terminalen Historien dieses Blocks betrachtet.
Die Definition eines Blocks hängt von der verwendeten Samplingstrategie ab. In dem Fall, dass Chance Sampling verwendet wird, enthält ein Block Q_j alle terminalen Historien mit derselben Sequenz von Zufallsereignissen.
Beim External Sampling hingegen wird ein Block durch eine Kombination aus Zufallsereignissen und Aktionen des Gegners bestimmt.
Die extremste hier genannte Form des Samplingverfahrens ist das Outcome Sampling, bei der Zufallsereignisse, Aktionen des Gegners sowie private Aktionen zufällig ausgewählt werden.

Sei $q_j > 0$ die Wahrscheinlichkeit, dass Block Q_j in der aktuellen Iteration ausgewählt wird. Für die Summe aller Blockwahrscheinlichkeiten gilt $\sum_{j=1}^{r} q_j = 1$.
Die Wahrscheinlichkeit, dass eine terminale Historie z in der aktuellen Iteration betrachtet wird, beträgt $q(z) = \sum_{j:z \in Q_j} q_j$.
Als nächstes wird im Paper eine gewichtete Form des Sampled Counterfactual Value eingeführt, um zu berücksichtigen, dass eine terminale Historie nur mit Wahrscheinlichkeit $q(z)$ in die Aktualisierung eingeht. Folgende Formel beschreibt den Sampled Counterfactual Value, wenn Block Q_j aktualisiert wird:

$$\tilde{v}_i(\sigma, I|j) = \sum_{z \in Q_j \cap Z_I} \frac{1}{q(z)} u_i(z) \pi_{-i}^{\sigma}(z[I]) \pi^{\sigma}(z[I], z). \tag{2.16}$$

([Lanctot et al., 2009, Eq. 6])
Die Gewichtung mit $\frac{1}{q(z)}$ sorgt dafür, dass der Erwartungswert des Sampled Counterfactual Value dem tatsächlichen Counterfactual Value entspricht:

$$\mathbb{E}_{j \sim q_j}[\tilde{v}_i(\sigma, I|j)] = v_i(\sigma, I). \tag{2.17}$$

([Lanctot et al., 2009, Lemma 1])

Diese Eigenschaft garantiert, dass MCCFR im Erwartungswert die gleichen Regret Updates wie CFR durchführt, obwohl nur ein Teil des Game Tree in jeder Iteration betrachtet

wird. Der MCCFR-Algorithmus sampelt in jeder Iteration einen Block und berechnet für jedes betroffene Informationsset die Sampled Immediate Counterfactual Regrets. Diese Regrets werden akkumuliert und die Strategie wird in der nächsten Iteration mittels Regret Matching aktualisiert.

2.3.2 Chance Sampling CFR

Chance Sampling CFR ist ein Spezialfall des MCCFR Frameworks, bei dem nur die Ausgänge von Zufallsereignissen gesampelt werden. Dies betrifft sowohl die Ausgänge von privaten als auch öffentlichen Zufallsereignissen. Nach dem Sampling wird der gesamte Subtree für diese Kombination traversiert, das heißt, alle möglichen Aktionen beider Spieler werden betrachtet.

2.3.3 External Sampling MCCFR

External Sampling MCCFR sampelt nur die Aktionen des Gegners sowie die Züge des Zufalls. Für jede deterministische Strategie τ von Gegner und Zufall wird ein Block $Q_\tau \in Q$ definiert. Dabei ist τ eine deterministische Abbildung von Informationssets $I \in \mathcal{I}_c \cup \mathcal{I}_{N\setminus\{i\}}$ auf Aktionen $A(I)$.

Die Blockwahrscheinlichkeiten sind:

$$q_\tau = \prod_{I \in \mathcal{I}_c} f_c(\tau(I)|I) \prod_{I \in \mathcal{I}_{N\setminus\{i\}}} \sigma_{-i}(\tau(I)|I). \tag{2.18}$$

Der Block Q_τ enthält alle terminalen Historien z, die mit τ konsistent sind (d. h. entlang von z wählt τ an allen $I \in \mathcal{I}_c \cup \mathcal{I}_{-i}$ jeweils die in z auftretende Aktion).

Es gilt $q(z) = \pi^\sigma_{-i}(z)$. Der Algorithmus iteriert über jeden Spieler $i \in N$ und traversiert für jeden den Spielbaum von der Wurzel bis zum Endknoten. An jeder History h mit $P(h) \neq i$ werden Aktionen gesampelt. Die gesampelten Aktionen werden pro Informationsset festgelegt, sodass innerhalb einer Iteration an allen Knoten desselben Informationssets dieselbe Aktion verwendet wird. In Spielen mit *Perfect Recall* wird ein Informationsset auf einem einzelnen Pfad höchstens einmal erreicht.

Für jedes besuchte Informationsset werden die Sampled Counterfactual Regrets berechnet:

$$\tilde{r}(I, a) = (1 - \sigma(a|I)) \sum_{z \in Q_\tau \cap Z_I} u_i(z) \pi_i^\sigma(z[I]a, z). \tag{2.19}$$

([Lanctot et al., 2009, Eq. 11])

2.3.4 Outcome Sampling MCCFR

Outcome Sampling MCCFR sampelt in jeder Iteration genau eine terminale Historie. Jeder Block $Q_j \in Q$ enthält eine terminale Historie, also $|Q_j| = 1$.

Die Samplingwahrscheinlichkeiten q_j spezifizieren eine Verteilung über terminale Historien. Diese wird über eine *Sampling Policy* σ' festgelegt, sodass $q(z) = \pi^{\sigma'}(z)$ gilt.

Damit die Gewichtung nicht ausartet, wird $q(z) \geq \delta > 0$ angenommen. Praktisch lässt sich dies durch Exploration sicherstellen, z. B. durch eine Untergrenze $\sigma'_i(a|I) \geq \epsilon > 0$.

In jeder Iteration wird eine terminale Historie $z \sim \sigma'$ gesampelt; aktualisiert werden nur die Informationssets entlang von z.

Während der Rückwärtstraversierung entlang von z werden die Sampled Counterfactual Regrets an jedem besuchten Informationsset berechnet und zu den Gesamtregrets hinzugefügt:

$$\tilde{r}(I,a) = \begin{cases} w_I \cdot (1 - \sigma(a|z[I])) & \text{falls } (z[I]a) \sqsubset z \\ -w_I \cdot \sigma(a|z[I]) & \text{sonst} \end{cases}, \tag{2.20}$$

wobei

$$w_I = \frac{u_i(z)\pi^{\sigma}_{-i}(z)\pi^{\sigma}_i(z[I]a, z)}{\pi^{\sigma'}(z)}. \tag{2.21}$$

([Lanctot et al., 2009, Eq. 10])

w_I ist der Importance-Sampling-Faktor für die Samplingwahrscheinlichkeit $\pi^{\sigma'}(z)$. Die Bedingung $(z[I]a) \sqsubset z$ bedeutet, dass auf der gesampelten Historie nach dem Präfix $z[I]$ die Aktion a gespielt wurde.

2.3.5 Theoretische Eigenschaften

Lanctot et al. [Lanctot et al., 2009] verwenden zur genaueren Angabe von Regretschranken die Größe M_i. Sie beschreibt die Komplexität eines Spiels feiner als die Anzahl der Informationssets und ist im Anhang definiert (Abschnitt 8.1).

Regret Bounds

Für Vanilla CFR gilt:

$$R_i^T \leq \Delta_{u,i} M_i \sqrt{|A_i|}/\sqrt{T}, \tag{2.22}$$

[Lanctot et al., 2009, Thm. 3]. mit $\Delta_{u,i} = \max_z u_i(z) - \min_z u_i(z)$ und $|A_i| = \max_{h:P(h)=i} |A(h)|$.

Für External Sampling MCCFR gilt für jedes p mit $0 < p \leq 1$ mit Wahrscheinlichkeit mindestens $1 - p$:

$$R_i^T \leq \left(1 + \sqrt{\frac{2}{p}}\right) \Delta_{u,i} M_i \sqrt{|A_i|}/\sqrt{T}. \tag{2.23}$$

[Lanctot et al., 2009, Thm. 4].

Obwohl External Sampling nur asymptotisch mehr Iterationen benötigt als Vanilla CFR, wird pro Iteration nur ein Bruchteil des Spielbaums traversiert. Für ausgewogene Spiele, in denen die Spieler ungefähr gleich viele Entscheidungen treffen, liegen die Iterationskosten (Anzahl besuchter Historien) bei External Sampling bei $O(\sqrt{|H|})$, während Vanilla CFR $O(|H|)$ benötigt. Damit ergibt sich asymptotisch eine geringere Gesamtzeit zur Berechnung eines approximativen Gleichgewichts [Lanctot et al., 2009, Sec. 4].

Für Outcome Sampling MCCFR gilt für jedes p mit $0 < p \leq 1$ mit Wahrscheinlichkeit mindestens $1 - p$:

$$R_i^T \leq \left(1 + \sqrt{\frac{2}{p}} \cdot \frac{1}{\sqrt{\delta}}\right) \Delta_{u,i} M_i \sqrt{|A_i|}/\sqrt{T}, \tag{2.24}$$

wenn $q(z) \geq \delta > 0$ für alle relevanten terminalen Historien z gilt [Lanctot et al., 2009, Thm. 5]. Der Faktor $1/\sqrt{\delta}$ hängt von der Sampling Policy ab.

2.4 CFR+

CFR+ wurde 2015 von Tammelin et al. [Tammelin et al., 2015] als Optimierung von CFR vorgestellt. Der Algorithmus erreicht durch drei Hauptänderungen eine deutlich schnellere Konvergenz als Vanilla CFR. CFR+ bildet die Grundlage für Cepheus, das erste Programm, welches Heads-up Limit Texas Hold'em erfolgreich gelöst hat [Bowling et al., 2015]. Zudem wurde CFR+ in Libratus eingesetzt, einer Poker-KI, die 2017 professionelle Spieler in Heads-up No-Limit Texas Hold'em besiegte [Brown and Sandholm, 2018].

Dieses Kapitel stellt die theoretischen Grundlagen vor: Regret Matching+ als zentrale Änderung, alternierende Updates und eine linear gewichtete Durchschnittsstrategie. Zudem werden die Tracking Regret Eigenschaften erläutert, die die praktische Überlegenheit von CFR+ erklären.

2.4.1 Unterschiede zu Vanilla CFR

CFR+ unterscheidet sich von Vanilla CFR durch drei Hauptänderungen.

Erstens verwendet CFR+ eine linear gewichtete Durchschnittsstrategie statt der uniformen Durchschnittsstrategie. Die gewichtete Durchschnittsstrategie ist definiert als $\bar{\sigma}_p^T = 2/(T^2 + T) \sum_{t=1}^{T} t\sigma_p^t$, wobei spätere Iterationen stärker gewichtet werden.

Zweitens führt CFR+ alternierende Updates durch. Im Gegensatz zu Vanilla CFR, das die Regrets beider Spieler simultan aktualisiert, aktualisiert CFR+ die Regrets der Spieler abwechselnd.

Drittens verwendet CFR+ Regret Matching+ anstelle von Regret Matching. Regret Matching+ setzt negative Regrets auf null zurück, statt sie zu ignorieren, was zu besseren Tracking Regret Eigenschaften führt.

2.4.2 Regret Matching+

Regret Matching+ ist ein Regret minimierender Algorithmus, der ähnlich zu Regret Matching operiert. Der entscheidende Unterschied liegt im Umgang mit negativen Regrets: Während Regret Matching Aktionen mit akkumuliertem negativen Regret ignoriert, setzt Regret Matching+ jeden akkumulierten negativen Regret auf null zurück.

Formal speichert Regret Matching+ nicht die Regrets $R^t(a)$, sondern verfolgt einen regret ähnlichen Wert:

$$Q^t(a) = \left(Q^{t-1}(a) + \Delta R^t(a)\right)^+, \tag{2.25}$$

wobei $x^+ = \max(x, 0)$ den positiven Anteil bezeichnet. Die Strategie basiert auf diesen Werten:

$$\sigma^t(a) = \frac{Q^{t-1}(a)}{\sum_{b \in A} Q^{t-1}(b)}. \tag{2.26}$$

([Tammelin et al., 2015])

Für Regret Matching+ gilt folgende Regret-Schranke ([Tammelin et al., 2015, Thm. 1]). Gegeben sei eine Menge von Aktionen A und eine Sequenz von T Wertfunktionen $v^t : A \to \mathbb{R}$. Es existiere eine Schranke L, sodass $|v^t(a) - v^t(b)| \leq L$ für alle t und $a, b \in A$ gilt. Ein Agent, der nach dem Regret Matching+ Algorithmus handelt, hat dann einen Regret von höchstens

$$R^T \leq L\sqrt{|A|T}. \tag{2.27}$$

([Tammelin et al., 2015, Thm. 1])

Diese Schranke ist identisch mit der Regret-Schranke von Regret Matching. CFR+ hat daher die gleiche theoretische Regret-Schranke wie CFR, konvergiert in der Praxis jedoch deutlich schneller. Dies lässt sich durch die besseren Tracking Regret Eigenschaften von Regret Matching+ erklären.

Die empirische Überlegenheit von Regret Matching+ hat eine intuitive Erklärung. Regret Matching+ übertrifft Regret Matching insbesondere dann, wenn sich die beste Aktion plötzlich ändert. Bei Regret Matching muss der Algorithmus warten, bis eine zuvor schlechte Aktion sich beweist, und muss dabei ihr gesamtes akkumuliertes negatives Regret überwinden. Regret Matching+ kann dagegen sofort die neue beste Aktion spielen, da negatives Regret auf null zurückgesetzt wird und nicht erst überwunden werden muss.

2.4.3 Tracking Regret

Tracking Regret ist ein von Herbster und Warmuth eingeführtes Konzept [Herbster and Warmuth, 1998]. Während Standard Regret nur gegen statische Strategien vergleicht, die immer die gleiche Aktion wählen, erlaubt Tracking Regret den Vergleich gegen Strategien, die ihre Aktion höchstens $(k-1)$ Mal ändern können.

Regret Matching hat sehr schlechte Tracking Regret Eigenschaften. Der Algorithmus kann lineares Regret haben, selbst wenn die Vergleichsstrategie nur einmal wechselt ($k = 2$). Im Gegensatz dazu ist Regret Matching+ der erste Regret Matching basierte Algorithmus mit sublinearem Tracking Regret.

Für alternative Strategien, die bis zu $(k-1)$ Mal wechseln können, hat Regret Matching+ eine Regret-Schranke von

$$R^T \leq kL\sqrt{|A|T}. \tag{2.28}$$

([Tammelin et al., 2015, Thm. 2]) Diese Schranke zeigt, dass der Regret linear mit der Anzahl der Wechsel k skaliert, aber sublinear mit der Anzahl der Iterationen T bleibt.

2.4.4 Linear Weighted Average

CFR+ verwendet eine linear gewichtete Durchschnittsstrategie anstelle der uniformen Durchschnittsstrategie von CFR. Die linear gewichtete Durchschnittsstrategie ist definiert als

$$\bar{\sigma}_p^T = \frac{2}{T^2 + T} \sum_{t=1}^{T} t\sigma_p^t, \tag{2.29}$$

wobei spätere Iterationen stärker gewichtet werden.

Die linear gewichtete Durchschnittsstrategie konvergiert zu einem ε-Nash Equilibrium ([Tammelin et al., 2015, Thm. 3]). Für eine Sequenz von T Verhaltensstrategieprofilen σ^t aus CFR+ in einem extensiven Spiel, wobei $\max |v_p(l) - v_p(l')| \leq L$ für alle Spieler p und terminale Historien l, l' gilt, ist die linear gewichtete Durchschnittsstrategie ein $2(|\mathcal{I}_1| + |\mathcal{I}_2|)L\sqrt{k}/\sqrt{T}$-Nash Equilibrium.

Die Schranke $\varepsilon = 2(|\mathcal{I}_1| + |\mathcal{I}_2|)L\sqrt{k}/\sqrt{T}$ impliziert eine Konvergenzrate von $O(1/\sqrt{T})$, die asymptotisch identisch mit der Rate von uniform weighting ist. Theorem 3 gilt nicht für CFR, und im Gegensatz zu CFR+, wo linear weighting die Performance verbessert, verschlechtert linear weighting die Performance von CFR. In der Praxis erreicht die aktuelle Strategie in CFR+ oft bereits eine gute Approximation zum Nash Equilibrium, sodass das aggressive Weighting die Konvergenz weiter beschleunigen kann.

2.5 Discounted CFR

Discounted CFR (DCFR) wurde 2018 von Brown und Sandholm [Brown and Sandholm, 2019a] als weitere Optimierung von CFR vorgestellt. Der Algorithmus übertrifft CFR+ in der Praxis häufig um einen Faktor von 2-3 und stellt damit den aktuellen Stand der Technik dar. DCFR basiert auf der Idee, frühere Iterationen zu *discounten*, wodurch neuere Iterationen stärker gewichtet werden. Dies beschleunigt die Konvergenz insbesondere in Spielen mit stark suboptimalen Aktionen, bei denen frühe Fehler in Vanilla CFR und CFR+ lange Zeit im kumulativen Regret erhalten bleiben.

2.5.1 Motivation

Ein zentrales Problem von CFR und CFR+ liegt darin, dass alle Iterationen gleich gewichtet werden. Betrachtet man ein einfaches Beispiel mit drei Aktionen mit Auszahlungen von 0, 1 und -1.000.000, so wählt CFR auf der ersten Iteration jede Aktion mit gleicher Wahrscheinlichkeit. Dies führt zu einem hohen Regret für die schlechte Aktion, der über viele Iterationen hinweg erhalten bleibt. In diesem Beispiel benötigt CFR+ 471.407 Iterationen, um die optimale Aktion mit 100% Wahrscheinlichkeit zu wählen. Durch das Discounting früher Iterationen kann DCFR dieses Problem adressieren und in derselben Situation bereits nach 970 Iterationen konvergieren [Brown and Sandholm, 2019a].

2.5.2 Formale Definition

DCFR wird durch drei Parameter charakterisiert: α für das Discounting positiver Regrets, β für das Discounting negativer Regrets, und γ für die Gewichtung der durchschnittlichen Strategie. Nach jeder Iteration t werden die kumulativen Regrets wie folgt aktualisiert:

$$R_t^+(I,a) = (R_{t-1}(I,a) + r_t(I,a)) \cdot \frac{t^\alpha}{t^\alpha + 1} \quad \text{falls } R_{t-1}(I,a) + r_t(I,a) \geq 0 \tag{2.30}$$

$$R_t^-(I,a) = (R_{t-1}(I,a) + r_t(I,a)) \cdot \frac{t^\beta}{t^\beta + 1} \quad \text{falls } R_{t-1}(I,a) + r_t(I,a) < 0 \tag{2.31}$$

wobei $r_t(I,a)$ der instantane Regret für Aktion a in Informationsset I in Iteration t ist. Für $\beta = 0$ wird der Faktor $\frac{t^0}{t^0+1} = \frac{1}{2}$ verwendet, was bedeutet, dass negative Regrets in jeder Iteration halbiert werden.

Die durchschnittliche Strategie wird mit Gewichtung t^γ akkumuliert:

$$\Sigma_t(I,a) = \Sigma_{t-1}(I,a) + \pi_t(I) \cdot \sigma_t(I,a) \cdot t^\gamma \tag{2.32}$$

wobei $\pi_t(I)$ die Wahrscheinlichkeit ist, Informationsset I in Iteration t zu erreichen, und $\sigma_t(I,a)$ die Wahrscheinlichkeit, Aktion a in Iteration t zu wählen.

2.5.3 Parameterwahl

Die empfohlene Parameterkonfiguration ist $\text{DCFR}_{3/2,0,2}$ mit $\alpha = 1.5$, $\beta = 0$ und $\gamma = 2$. Diese Variante übertrifft CFR+ in allen getesteten Spielen, einschließlich großskaliger realistischer Szenarien wie Heads-up No-Limit Texas Hold'em Subgames [Brown and Sandholm, 2019a].

Für $\beta = 0$ nähern sich negative Regrets einem konstanten Wert an, anstatt gegen $-\infty$ zu gehen. Dies kann die Kompatibilität mit Pruning-Algorithmen beeinträchtigen, die auf negativen Regrets basieren. Für solche Anwendungen kann $\text{DCFR}_{3/2,1/2,2}$ mit $\beta = 0.5$ verwendet werden, was negative Regrets gegen $-\infty$ konvergieren lässt und damit Pruning ermöglicht.

2.5.4 Theoretische Eigenschaften

DCFR behält die gleiche asymptotische Konvergenzrate wie CFR von $O(1/\sqrt{T})$, unterscheidet sich jedoch durch einen konstanten Faktor. Theorem 2 aus [Brown and Sandholm, 2019a] zeigt, dass die gewichtete Durchschnittsstrategie nach T Iterationen ein $6\Delta|I|(\sqrt{|A|} + 1/\sqrt{T})/\sqrt{T}$-Nash-Gleichgewicht bildet, wobei Δ der Payoff-Bereich, $|I|$ die Anzahl der Informationssets und $|A|$ die maximale Anzahl an Aktionen pro Informationsset ist. Trotz dieser theoretischen Garantie konvergiert DCFR in der Praxis deutlich schneller als CFR+.

2.6 Exploitability einer Strategie

Um die Qualität einer Strategie zu bewerten, ist es wichtig, ihre Worst-Case-Performance zu kennen. In Nullsummenspielen kann dies durch die Berechnung einer *best response* ermittelt werden. Eine best response ist die optimale Strategie für einen Spieler gegen eine gegebene Gegnerstrategie.

Die *exploitability* einer Strategie misst, wie viel zusätzlicher Nutzen durch eine best response verloren geht, verglichen mit einer optimalen Strategie. In einem Zweipersonen-Nullsummenspiel ist die exploitability einer Strategie σ_i definiert als

$$\varepsilon_i(\sigma_i) = v_i - u_i(\sigma_i, b_{-i}(\sigma_i)), \tag{2.33}$$

wobei v_i der Spielwert für Spieler i ist und $b_{-i}(\sigma_i)$ die best response des Gegners gegen σ_i darstellt. Eine Strategie ist optimal, wenn ihre exploitability null ist.

Für Strategien, die in beiden Positionen gespielt werden, wird die exploitability als Durchschnitt der best response Werte aus beiden Positionen berechnet:

$$\varepsilon(\sigma) = \frac{u_2(\sigma_1, b_2(\sigma_1)) + u_1(b_1(\sigma_2), \sigma_2)}{2}. \tag{2.34}$$

In Nullsummenspielen ist die exploitability eng mit dem Nash-Gleichgewicht verbunden: Eine Strategie ist optimal, wenn ihre exploitability null ist. Daher ist die exploitability eine zentrale Metrik zur Evaluation von Strategien.

2.6.1 Konventionelle Best Response Berechnung

Die konventionelle Berechnung einer best response erfolgt durch eine rekursive Traversierung des Information Set Trees. Der Algorithmus traversiert den Information Set Tree des betrachteten Spielers und berechnet für jedes Informationsset den erwarteten Nutzen. An Terminalknoten müssen alle möglichen Game States berücksichtigt werden, die dem Spieler nicht unterscheidbar sind. Dazu werden die *reach probabilities* des Gegners berechnet, also die Wahrscheinlichkeiten, mit denen der Gegner verschiedene Informationssets erreicht. Der Nutzen wird als gewichtete Summe über alle möglichen Game States berechnet, wobei jeder Nutzen mit der entsprechenden reach probability gewichtet wird. An Entscheidungsknoten wählt der Algorithmus die Aktion mit dem höchsten erwarteten Wert, um die best response Strategie zu konstruieren.

Diese Methode erfordert eine vollständige Traversierung des Information Set Trees, was für große Spiele wie Poker mit 10^{18} Game States selbst bei moderner Hardware nicht durchführbar ist.

2.6.2 Accelerated Best Response

Johanson et al. [Johanson et al., 2011] präsentieren eine beschleunigte Methode, die eine vollständige Traversierung vermeidet. Der Accelerated Best Response Algorithmus nutzt die Struktur von Information und Utilities, um Berechnungen wiederzuverwenden und zu vermeiden. Dies ermöglicht es erstmals, die Worst-Case-Performance von Strategien in großen Spielen wie Texas Hold'em zu berechnen.

2.6.3 Public State Tree

Der zentrale Ansatz besteht darin, statt des Information Set Trees einen *public state tree* zu traversieren. Ein *public state* ist definiert durch alle Informationen, die beiden Spielern bekannt sind, also wie das Spiel für einen Beobachter ohne private Informationen aussieht. Der public state tree ist deutlich kleiner als der Information Set Tree, da er nur die öffentlich sichtbaren Informationen enthält.

Bei der Traversierung des public state trees werden die reach probabilities des Gegners einmal berechnet. Diese werden für alle eigenen Informationssets in diesem public state wiederverwendet. Dies ermöglicht eine erhebliche Reduzierung der Strategieabfragen. In Spielen wie Texas Hold'em, wo jeder Spieler bis zu 1326 Informationssets pro public state haben kann, werden dadurch 1325 unnötige Strategieabfragen vermieden.

2.6.4 Effiziente Terminal-Node-Evaluierung

Ein weiterer Beschleunigungsschritt betrifft die Evaluierung von Terminalknoten. Eine naive Methode würde für jedes Paar von Informationssets den Nutzen berechnen, was eine $O(n^2)$ Komplexität bedeutet, wobei n die Anzahl der Informationssets pro Spieler ist.

In Pokerspielen kann diese Komplexität auf $O(n)$ reduziert werden, indem die Informationssets nach ihrer Handstärke sortiert werden. Da der Nutzen nur von der relativen Rangordnung der Hände abhängt, kann die Evaluierung durch geschickte Indizierung effizient durchgeführt werden. Für abhängige Verteilungen, wie sie in Poker auftreten, wird das Inklusions-Exklusions-Prinzip verwendet, um die korrekte Berechnung zu gewährleisten.

2.6.5 Game-Spezifische Isomorphismen

Ein weiterer Beschleunigungsschritt nutzt strategisch äquivalente Aktionen oder Ereignisse. In vielen Kartenspielen sind Karten mit gleichem Rang, aber unterschiedlicher Farbe strategisch äquivalent. Beispielsweise ist eine 2 Herz äquivalent zu einer 2 Pik, zumindest bis weitere Karten aufgedeckt werden. Solche Mengen äquivalenter Chance-Aktionen werden als *isomorph* bezeichnet, und eine wird willkürlich als kanonisch gewählt.

Wenn die zu evaluierende Strategie für jedes Mitglied jeder Menge isomorpher Historien gleich ist, kann die Größe des public state trees erheblich reduziert werden, indem nur kanonische Aktionen betrachtet werden. Beim Zurückkehren durch einen Chance-Knoten während der Traversierung muss der Nutzen einer kanonischen Aktion mit der Anzahl der isomorphen States gewichtet werden, die sie repräsentiert. In Texas Hold'em führt diese Reduktion zu einem public state tree, der 21.5-mal kleiner ist als das vollständige Spiel.

2.6.6 Parallele Berechnung

Ein vierter Beschleunigungsschritt nutzt die Möglichkeit, unabhängige Teilbäume des public state trees parallel zu lösen. Da der Graph der public states einen Baum bildet, können zwei public states, von denen einer kein Nachfahre des anderen ist, parallel gelöst werden, da sie keine gemeinsamen Nachfahren und somit keine gemeinsamen Berechnungen haben.

Beispielsweise können beim Erreichen eines öffentlichen Chance-Ereignisses während der Traversierung alle Kinder parallel gelöst werden. Der Wertvektor wird wie üblich zurückgegeben, sobald jede Teilbaum-Berechnung abgeschlossen ist.

In Texas Hold'em ist eine natürliche Wahl für unabhängige Teilspiele der Beginn der zweiten Runde, der *Flop*. Es gibt 7 nicht-terminale Aktionssequenzen in der ersten Runde und 1755 kanonische öffentliche Chance-Ereignisse zu Beginn des Flops, was zu 12.285 unabhängigen Teilspielen pro Position führt, insgesamt 24.570 Teilspiele.

2.7 Hold'em Games

In diesem Kapitel stelle ich vor, welche Pokervarianten im Rahmen dieser Arbeit mittels CFR gelöst werden sollen. Hierbei erkläre ich zunächst grundlegend wodurch sich die Untergruppierung von Pokerspielen, Hold'em Spiele, auszeichnen und stelle dann einige bekannte Varianten vor. Denn, wenn, man die Grundstruktur von Hold'em Spielen definiert hat, ist es einfach diese Struktur zu erweitern und alle folgenden Poker Hold'em Varianten darzustellen. Anschließend beschreibe ich die Varianten welche in dieser Arbeit gelöst werden und warum ich diese Auswahl getroffen habe.

Ich beschäftige mich in dieser Arbeit ausschließlich mit Heads-Up Limit Versionen von Hold'em Spielen. Heads-Up bedeutet, dass das Spiel nur zwei Teilnehmer beinhaltet. Die Klassifikation Limit bezieht sich darauf, dass die Setzgrößen der Spieler auf vorher festgelegte Beträge beschränkt sind. Diese zwei Einschränkungen reduzieren die Komplexität enorm. Die folgende Definition gilt für jegliche Heads-Up Limit Hold'em Spiele [Shi and Littman, 2001].

Eine Iteration eines Hold'em Spiels nennt man eine Hand. Eine Hand widerum besteht aus mehreren Runden. In der ersten Runde wird eine bestimmte Anzahl an privaten Karten (Hole Cards) an die zwei Teilnehmer verteilt. In *jeder* Runde wird eine bestimmte Anzahl (0 auch möglich), an öffentlichen Karten (Board Cards) aufgedeckt.

Nachdem Karten ausgeteilt oder aufgedeckt wurden, folgt immer eine Setz Runde, in welcher die Spieler abwechselnd Entscheidungen treffen. Ihnen stehen grundlegend 3 Auswahlmöglichkeiten zur Verfügung: "fold, call und raise„. Die Begrifflichkeiten können sich abhängig von der Quelle und dem Spiel etwas unterscheiden.
Wählt ein Spieler die Aktion *fold*, endet die Hand sofort und der andere Spieler gewinnt den Pot. Der Pot ist die Ansammlung aller Einsätze die im Verlauf der Hand gesetzt wurden.
Bei einem *call*, setzt er so viele Chips in den Pot, bis er genauso viele Chips in den Pot eingezahlt hat wie der andere Spieler. Die Anzahl an Chips kann auch 0 betragen. In Pokervarianten welche zu Unterhaltungs und nicht zu Forschungszwecken gespielt werden, nennt man das dann *check*.
Bei der Aktion *raise*, setzt der Spieler zunächst genauso viel Chips in den Pot wie sein Gegner, analog zum *call*. Anschließend setzt er dann zusätzlich noch einen durch die Spielregeln festgelegten Betrag an Chips.

Die Spieler treffen solange abwechselnd Entscheidungen bis entweder ein Spieler folded, in diesem Fall endet die Hand sofort, oder bis ein Spieler called (solange es nicht die erste Aktion der Runde ist). In dem Fall das ein Spieler called, endet nur die Runde und nicht die Hand. Um eine endlose Abfolge aus Raises zu vermeiden, gibt es eine Obergrenze für die Anzahl an Raises nacheinander ausgeführt werden dürfen. Wenn das Setzlimit erreicht ist, bleiben dem Spieler welcher am Zug ist, nur die Optionen Call und Fold. Wenn bis zum Ende der finalen Setzrunde kein Spieler gefolded hat, kommt es zu einem *Showdown*. Bei einem Showdown offenbaren beide Spieler ihre Hole Cards und bilden aus allen zu Verfügung stehenden Karten (Hole + Board Cards) die bestmögliche Pokerhand. Der Spieler mit der stärkeren Hand gewinnt den Pot, welche möglichen Pokerhände es gibt, und welcher Rangordnung diese folgen, hängt von der Variante des Spiels ab. Eine Übersicht über die verschiedenen Pokerhände und deren Bedeutung findet sich unter `https://en.wikipedia.org/wiki/List_of_poker_hands`. [Southey et al., 2005]

Da wir jetzt eine grundlegende Definition eines Heads-Up Limit Hold'em Spiels haben, können wir nun, durch anpassen einzelner Parameter jegliche Varianten definieren. Die relevantesten Aspekte sind: Die Anzahl der Hole und Board Cards, die Anzahl der Setzrunden, dem Setzlimit pro Runde, den im Deck enthaltenden Karten, die möglichen Pokerhände, die festgelegten Setzgrößen und noch einiges mehr.

2.7.1 Kuhn Poker

Die simpelste definierte Pokervariante ist One Card Poker auch Kuhn Poker gennant. Der Name Kuhn Poker stammt durch den Erfinder dieses Spiels Harald Kuhn, welcher es in seinem Paper aus dem Jahr 1951 mit dem Titel "Simplified Two-Person Poker „beschrieb.
Kuhn Poker wird mit einem Kartendeck bestehend aus 3 Karten gespielt. In seinem Paper sind die Karten mit 1,2 und 3 bennant, wobei 1 die schlechteste und 3 die beste Karte ist. Für diese Arbeit wird die Notation J, Q, K verwendet, die den Rängen eines Standard Kartenspiels entspricht.

Zu Beginn zahlen beide Spieler einen Pflichteinsatz in Höhe eines Chips in den Pot, dies wird auch Ante genannt. Beide Spieler erhalten eine private Karte und es werden keine öffentlichen Karten ausgeteilt. Nach dem austeilen der privaten Karte startet die erste und einzige Setzrunde, in welcher die Spieler jeweils einen Chip in den Pot setzen dürfen, womit das Setzlimit bei Eins liegt. Im Falle eines Showdowns, gewinnt die höhere Karte $J < Q < K$.

Kuhn definiert in seinem Paper auch noch andere Versionen seines Spiels indem er das Verhältnis zwischen Ante und Setzgröße anpasst. Der eben beschriebene Fall in welchem der Ante genauso groß ist wie die Setzgröße, wird von ihm Fall 2 genannt. Fall 1 liegt vor, wenn die Setzgröße kleiner als der Ante ist. Fall 3 tritt ein, wenn der Ante kleiner als die Setzgröße ist und die Setzgröße kleiner als das Doppelte des Antes ist. Fall 4 liegt vor, wenn die Setzgröße dem Doppelten des Antes entspricht. [Kuhn, 1951]

2.7.2 Leduc Hold'em

Leduc Hold'em [Southey et al., 2005] ist eine vereinfachte Hold'em Variante für zwei Spieler.

Leduc Hold'em wird mit einem Kartendeck bestehend aus zwei Suits mit drei Karten pro Suit gespielt, insgesamt sechs Karten. Die Karten Notation verwendet Rank und Suit, beispielsweise Js, Jh, Qs, Qh, Ks, Kh.

Das Spiel besteht aus zwei Runden. In der ersten Runde erhält jeder Spieler eine private Karte. In der zweiten Runde wird eine öffentliche Board Karte aufgedeckt.

Zu Beginn zahlen beide Spieler einen Ante von einem Chip in den Pot. Es gibt ein Maximum von zwei Bets pro Runde. Die Setzgrößen betragen 2 Chips in der ersten Runde und 4 Chips in der zweiten Runde.

Die möglichen Pokerhände sind entweder ein Paar oder nur eine High Card. Ein paar gewinnt gegen eine High Card.

2.7.3 Zwölf Karten Poker

Die folgende Pokervariante ist von mir selbst entworfen worden, als eine Zwischenstufe der Komplexität zwischen Leduc Hold'em und Rhode Island Hold'em. Die 12 Karten Variante verwendet ein Deck mit zwölf Karten: J, Q, K und A in drei Suits (s, h, d).

Es gibt drei Runden. In der ersten Runde erhält jeder Spieler eine private Karte. In der zweiten und dritten Runde wird jeweils eine öffentliche Board Karte aufgedeckt.

Es gibt ein Maximum von zwei Bets pro Runde. Die Setzgrößen betragen 2 Chips in der ersten Runde, 4 Chips in der zweiten Runde und 8 Chips in der dritten Runde. Zu Beginn zahlen beide Spieler einen Ante von 1 Chip in den Pot.

Das Spiel endet nach der dritten Runde mit einem Showdown. Die Hand besteht aus der Kombination der privaten Karte des Spielers und den beiden öffentlichen Board Karten. Die Hand Rankings sind: Three of a Kind schlägt ein Paar, ein Paar schlägt High Card.

2.7.4 Rhode Island Hold'em

Rhode Island Hold'em [Shi and Littman, 2001] wurde von Shi und Littman im Jahr 2001 als Testbed für AI Forschung entwickelt.

Rhode Island Hold'em verwendet ein Standard Kartendeck mit 52 Karten. Zu Beginn zahlen beide Spieler einen Ante von 5 Chips in den Pot. Jeder Spieler erhält eine private Karte.

Das Spiel besteht aus drei Runden. Die erste Runde findet nach dem Austeilen der privaten Karten statt. Die Bet Size beträgt 10 Chips und es gibt ein Limit von drei Raises pro Runde. Nach der ersten Runde wird eine öffentliche Board Karte, der Flop, aufgedeckt. Die zweite Runde findet nach dem Flop statt. Die Bet Size beträgt 20 Chips. Nach der zweiten Runde wird eine weitere öffentliche Board Karte, der Turn, aufgedeckt. Die dritte und finale Runde findet nach dem Turn statt. Die Bet Size beträgt ebenfalls 20 Chips.

Beim Showdown bildet jeder Spieler die beste 3 Card Poker Hand aus seiner privaten Karte und den beiden öffentlichen Board Karten. Die Hand Rankings für 3 Card Poker unterscheiden sich von 5 Card Poker. Three of a Kind schlägt Straight und Flush. Straight schlägt Flush. Flush schlägt Pair und Pair schlägt High Card. Bei einem Unentschieden wird der Pot gleichmäßig geteilt.

2.7.5 Limit Texas Hold'em

Texas Hold'em [Southey et al., 2005] ist das häufigste Format für Hold'em.

Texas Hold'em verwendet ein Standard Kartendeck mit 52 Karten, bestehend aus 2 bis Ass in vier Suits.

Das Spiel besteht aus vier Betting Runden. In der ersten Runde erhalten die Spieler jeweils zwei private Karten. In der zweiten Runde, dem Flop, werden drei Board Karten

aufgedeckt. In der dritten Runde, dem Turn, wird eine vierte Board Karte aufgedeckt. In der vierten Runde, dem River, wird eine fünfte Board Karte aufgedeckt.

Es gibt ein Maximum von vier Bets pro Runde. Die festgelegten Bet Sizes betragen 10 Chips in den ersten beiden Runden und 20 Chips in den letzten beiden Runden. In diesem Variation beginnt das Spiel nicht mit einem Ante, bei dem beide Spieler die gleiche Anzahl an Chips in den Pot zahlen, sondern mit einer Struktur die man Blinds nennt. Dabei zahlt der erste Spieler 5 Chips in den Pot und der zweite 10 Chips.

Die möglichen Pokerhände sind Royal Flush, Straight Flush, Four of a Kind, Full House, Flush, Straight, Three of a Kind, Two Pair, One Pair und High Card. Diese Vielfalt an möglichen Händen trägt zur hohen Komplexität des Spiels bei.

Kapitel 3

Analyse

In diesem Kapitel setze ich einen Rahmen für den Implementierungsteil dieser Arbeit. Dies umfasst die Spielvarianten welche gelöst werden sollen, die zur Lösung verwendeten CFR-Varianten, sowie Evaluationsmethoden die genutzt werden um die Strategiequalität zu bewerten. Ich werde ausserdem kurz darauf eingehen welche Anforderungen an die Implementierung ich ursprünglich in meinem Exposé gestellt habe und die Unterschiede zu dem was letzendlich umgesetzt wurde.

3.1 Spielvarianten

Ich habe im Rahmen meines Exposés bereits eine grobe Schätzung darüber abgegeben, welche Pokervarianten ich lösen zu können glaube. Dies beinhaltet zuerst das wohl einfachste Pokerspiel, Kuhn Poker. Anschließend die häufig genutze Form Leduc Hold'em, welche genau wie Kuhn Poker, ausschließlich zu Forschungszwecken verwendet wird. Das dritte Spiel habe ich mir selbst ausgedacht und "Twelve Card Poker"getauft. Dies geschah, da mir zu diesem Punkt noch keine weiteren Forschungsvarianten bekannt waren und ich eine weitere Komplexitätsstufe benötigte. Im Verlauf meiner Arbeit habe ich die Varianten Rhode Island Hold'em und Royal Hold'em kennen gelernt und ebenso umgesetzt. Zusätzlich wurde auch Limit Hold'em implementiert, welches die höchste Komplexität aller in dieser Arbeit Implementierten Varianten aufweist.

3.2 CFR Solver

In dieser Arbeit werden drei verschiedene Varianten von CFR Algorithmen implementiert: Vanilla CFR, CFR+ und MCCFR mit External Sampling. Die Auswahl dieser drei Varianten erfolgte zu Beginn der Arbeit basierend auf einer ersten Literaturrecherche. Diese Varianten wurden in der mir bekannten Literatur am häufigsten diskutiert und bilden die Grundlage für viele erfolgreiche Anwendungen.
Vanilla CFR dient als Baseline für den Vergleich und ermöglicht es festzustellen, inwiefern sich die Ergebnisse der einzelnen Algorithmen unterscheiden. MCCFR wurde ausgewählt, um eine Variante zu untersuchen, die nicht den gesamten Game Tree explizit

besucht, sondern mittels Sampling arbeitet. Als drittes wird CFR+ umgesetzt, da es die Grundlage für extrem erfolgreiche Poker KIs wie Cepheus oder Libratus bildet.
Zum Zeitpunkt der anfänglichen Literaturrecherche war mir noch nicht bekannt was für ein großes Maß an unterschiedlichen CFR Algorithmen existiert. Im Nachhinein betrachtet wäre es durchaus sinnvoll gewesen eine andere Wahl an Optmierungsalgorithmen zu treffen und Varianten wie zum Beispiel BR-CFR oder Discounted CFR zu untersuchen.

3.3 Evaluationsmethoden

Ursprünglich war im Exposé die Evaluation mittels Exploitability Tests mit einem Best Response Agent geplant. Diese Methode ermöglicht es, die Qualität der trainierten Strategien zu bewerten, indem ein optimaler Gegner simuliert wird, welcher die Strategie maximal ausnutzt. Tatsächlich wurde in der Arbeit ein Accelerated Best Response Agent implementiert, welcher das gleiche Ziel verfolgt. Allerdings beinhaltet dieser einige Performance verbessernde Änderungen. Um das Konvergenzverhalten eines Algorithmus einschätzen zu können, wird während des Trainings alle X Iterationen die Exploitability bestimmt. Nach dem Training wird diese dann im Vergleich zu der Anzahl der Iterationen oder der vergangenen Zeit in einem Diagramm dargestellt.
Eine weitere Metrik ist der Erwartungswert einer Strategie im Self-Play. Hierbei spielt eine Strategie wiederholt gegen sich selbst, wodurch der Erwartungswert des Spiels berechnet wird. Bei Forschungsvarianten wie Leduc Hold'em ist dieser Wert bekannt und kann daher zur Verifikation der Implementierung genutzt werden.

In diesem Kapitel wurden die Komponenten dieser Arbeit vorgestellt und die Anforderungen an diese definiert. Es wurden die zu untersuchenden Spielvarianten, die zu implementierenden CFR Algorithmen und die verwendeten Evaluationsmethoden dargelegt. Im folgenden Kapitel beschreibe ich wie die Architektur der Software aussieht.

Kapitel 4

Architektur und Design

In diesem Kapitel wird die Architektur des Softwareprojekts beschrieben. Hierbei wird zuerst ein Überblick darüber gegeben, wie die einzelnen Bestandteile miteinander interagieren. Anschließend gehe ich auf die Struktur der einzelnen Komponenten ein und stelle die wichtigsten von ihnen mit UML Diagrammen dar. Ich erkläre wichtige Designentscheidungen und erwähne Probleme die dadurch entstanden sind. Die Probleme und deren Konsequenzen werden anschließend im Kapitel 5 Implementierung genauer erläutert.

Eine zentrale Designentscheidung, welche schon zu Beginn getroffen wurde, ist eine explizite Trennung der Pokerspiellogik vom CFR Algorithmus. Die Logik der Pokerspiele, zukünftig Game Environment genannt, wird als separate Entität implementiert, welche das Spielverhalten und die Zustandsverwaltung übernimmt. Ziel dieser Unterteilung war es, das Projekt später einfach erweiterbar für neue Spiele und Algorithmen zu halten.

Dieses Kapitel gliedert sich in drei Hauptkomponenten: Game Environment, CFR Solver und Evaluationsmethoden.

4.1 Game Environment

In diesem Abschnitt wird der Aufbau der Spielumgebung mithilfe eines Klassendiagramms beschrieben. Die Spielumgebung ist für die Logik des Pokerspiels sowie für die Verwaltung des Spielzustands zuständig. Eine Spielumgebung besteht aus den folgenden Komponenten: `Player` (repräsentiert die beiden Spieler), `Dealer` (verwaltet das Kartendeck), `Judger` (bestimmt den Gewinner) und `Game` (koordiniert die anderen Komponenten). Bei allen Varianten außer Kuhn Poker wird diese Architektur zusätzlich durch eine `Round` Klasse erweitert, welche den Übergang zwischen Setzrunden steuert. Diese Architektur wurde von RLCard übernommen `https://rlcard.org`. a

Abbildung 4.1 zeigt das Klassendiagramm der Game Environment für die Pokervarianten Kuhn Poker und Leduc Hold'em. Alle weiteren Spielvarianten basieren auf Leduc Hold'em und fügen zusätzliche Funktionalitäten hinzu.

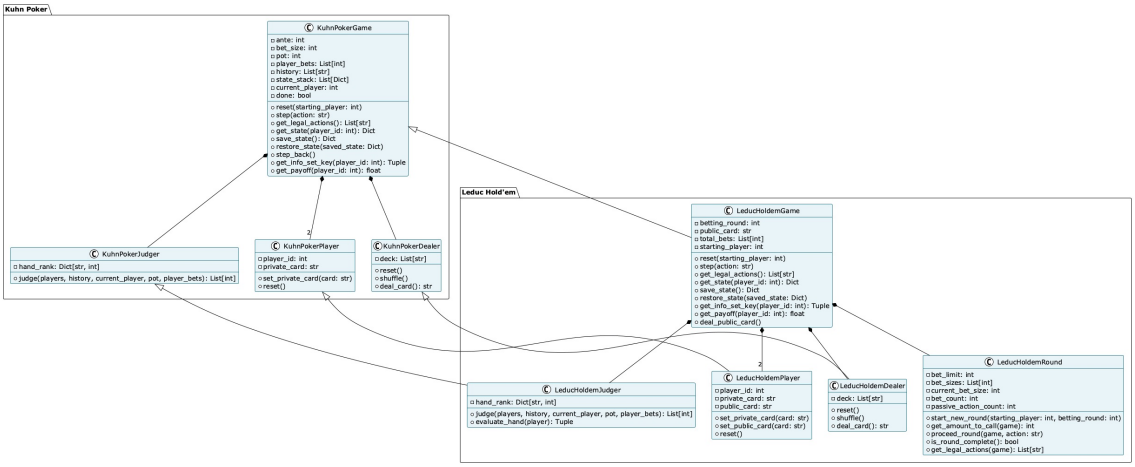

Abbildung 4.1: Klassendiagramm der Game Environment für Kuhn Poker und Leduc Hold'em

4.2 CFR Solver

Dieses Kapitel ist unterteilt in 2 Abschnitte, aufgrund von verschiedenen Designs welche genutzt wurden um einen CFR Algorithmus zu implementieren. Das erste Design nutzt die öffentlichen Methoden der Game Environment um während des Trainings den Spielbaum dynamisch aufzubauen und gleichzeitig zu durchlaufen. Das zweite Design verwendet einen statischen Spielbaum, welcher im Voraus mithilfe der Game Environment gebaut und anschließend auf der Festplatte gespeichert wurde.

4.2.1 Dynamischer Spielbaum

Die dynamische Implementierung des CFR Solvers verzichtet auf die Speicherung eines expliziten Spielbaums. Stattdessen wird der Spielbaum bei jeder Iteration und für jede Kartenkombination neu durch wiederholte Aufrufe von `game.step()` und `game.step_back()`

traversiert. Dieser Ansatz benötigt weniger Arbeitsspeicher als die statische Variante, da nicht der gesamte Baum im Speicher gehalten werden muss. Allerdings ist er durch wiederholtes Aufrufen der Spielzustandsfunktionen deutlich langsamer.

Abbildung 4.2 zeigt das Klassendiagramm der dynamischen CFR Solver Implementierung.

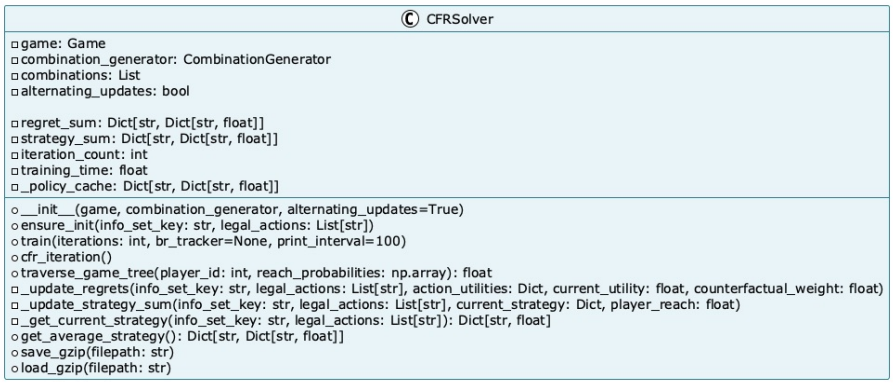

Abbildung 4.2: Klassendiagramm der dynamischen CFR Solver Architektur

Der Trainingsprozess beginnt mit der Initialisierung der Objekte Game, Combination Generator und Solver. Anschließend erzeugt der Combination Generator alle möglichen Kartenkombinationen.
Eine Iteration des CFR Algorithmus enthält dann folgende Schritte:
Für jede Kartenkombination welche vom Combination Generator generiert wurde, wird der Spielbaum rekursiv durchlaufen. Hierbei werden alle möglichen Aktionssequenzen der beiden Spieler betrachtet. Auf die genaue Funktionsweise der rekursiven Traversierung gehe ich in der kommenden Seite ein.
Nachdem der Spielbaum für alle Kartenkombinationen und Aktionen traversiert wurde, wird die Strategie für alle Informationssets mittels Regret Matching basierend auf den kumulierten Counterfactual Regrets aktualisiert.
Hiermit endet eine Iteration.
Optional kann während des Trainings eine Exploitability Berechnung durchgeführt werden. Hierfür wird die durchschnittliche Strategie berechnet und anschließend eine Best Response Evaluation durchgeführt.
Dieser Prozess wird für die gewünschte Anzahl an Iterationen wiederholt und anschließend wird das Modell gespeichert.

Wichtig zu erwähnen ist, dass die Best Response Evaluation nicht bei jeder Iteration durchgeführt wird, sondern in regelmäßigen Abständen gemäß einer konfigurierbaren Evaluationsfrequenz. Diese Abstände werden so gewählt, dass aus den gesammelten Werten anschließend ein aussagekräftiger Konvergenzplot erstellt werden kann.

Auf der folgenden Seite wird der eben wörtlich beschriebende Ablauf mithilfe eines UML Aktivitätsdiagramms veranschaulicht.

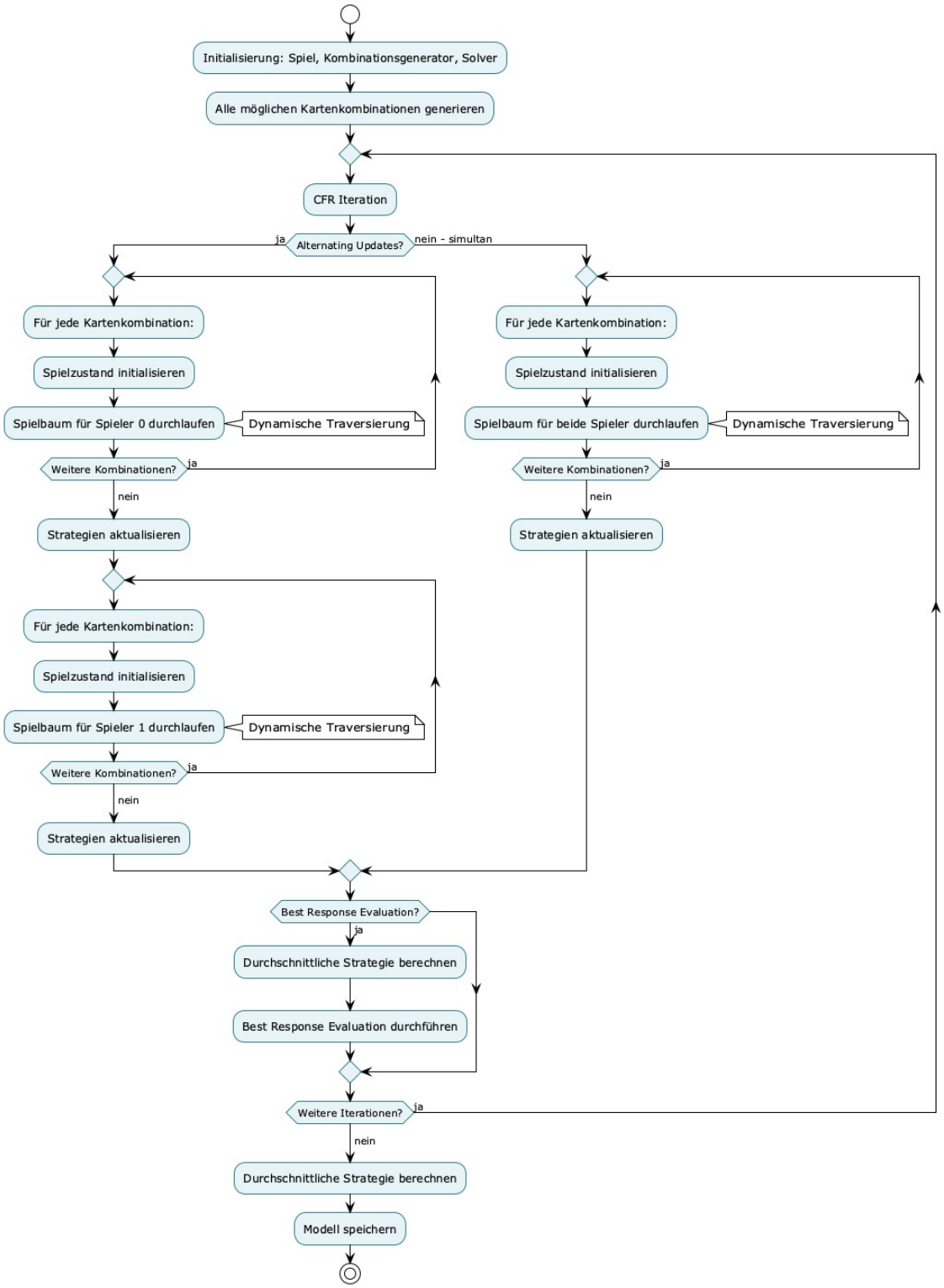

Abbildung 4.3: Aktivitätsdiagramm des Trainingsprozesses der dynamischen CFR Solver Implementierung

Abbildung 4.4 zeigt den Ablauf der rekursiven Spielbaumtraversierung im Detail.

In diesem Abschnitt wird die Rekursive Spielbaumtraversierung genauer beschrieben.

Die Traversierungsfunktion wird im Trainingsablauf für jede Kartenkombination aufgerufen, wobei für jeden Kartendeal der Spielbaum dynamisch durchlaufen wird. Die Funktion berechnet den erwarteten Nutzen für einen gegebenen Spieler und aktualisiert dabei die Counterfactual Regrets. Wichtig ist, dass die Regrets nur für den Spieler aktualisiert werden, für den traversiert wird, während die Strategie des Gegners lediglich zur Berechnung des erwarteten Nutzens verwendet wird. Als Parameter erhält die Funktion die Spieler-ID für den zu traversierenden Spieler und die Erreichbarkeitswahrscheinlichkeiten beider Spieler. Die Erreichbarkeitswahrscheinlichkeit beschreibt die Wahrscheinlichkeit, den aktuellen Knoten zu erreichen. Beim initialen Aufruf der Traversierungsfunktion werden für beide Spieler Erreichbarkeitswahrscheinlichkeiten von 1.0 übergeben, da beide Spieler den Root Node mit Sicherheit erreichen.

Die Traversierung beginnt mit einer Prüfung, ob es sich um einen Terminal Node handelt. Falls dies der Fall ist, wird der Nutzen für den betrachteten Spieler berechnet und zurückgegeben. Falls die Erreichbarkeitswahrscheinlichkeit null ist, wird sofort null zurückgegeben.

Handelt es sich um einen **Gegner-Knoten**, wird die aktuelle Strategie für das entsprechende Informationsset mittels Regret Matching bestimmt. Es wird zuerst eine legale Aktion ausgeführt und die Erreichbarkeitswahrscheinlichkeiten aktualisiert. Anschließend wird rekursiv die Traversierungsfunktion aufgerufen, welche den gesamten Teilbaum traversiert, bis Terminal Nodes erreicht werden, und den erwarteten Nutzen für diesen Teilbaum zurückgeben. Nachdem der rekursive Aufruf abgeschlossen ist und den erwarteten Nutzen zurückgegeben hat, wird die Aktion rückgängig gemacht und der Prozess wird für die nächste Aktion wiederholt. Nachdem alle Aktionen verarbeitet wurden, wird der erwartete Nutzen über alle Aktionen berechnet und zurückgegeben.

Handelt es sich um einen **Spieler-Knoten**, wird ebenfalls die aktuelle Strategie für das entsprechende Informationsset bestimmt. Analog zum Gegner-Knoten wird für jede mögliche Aktion die Aktion ausgeführt, die Erreichbarkeitswahrscheinlichkeiten werden aktualisiert, und rekursiv wird die Traversierungsfunktion aufgerufen. Nachdem der rekursive Aufruf abgeschlossen ist und den erwarteten Nutzen zurückgegeben hat, wird die Aktion rückgängig gemacht. Im Gegensatz zum Gegner-Knoten wird hier jedoch der Nutzen jeder Aktion gespeichert, da dieser für die anschließende Regret-Berechnung benötigt wird. Nachdem alle Aktionen verarbeitet wurden, wird der erwartete Nutzen über alle Aktionen berechnet. Anschließend wird für jede Aktion das Counterfactual Regret berechnet. Das kumulierte Regret und die kumulierte Strategie werden für das Informationsset aktualisiert und darauffolgend equivalent zum Gegner Knoten der erwartete Nutzen zurückgegeben.

Auf der folgenden Seite wird der Ablauf der rekursiven Traversierung mit einem UML Aktivitätsdiagramm beschrieben

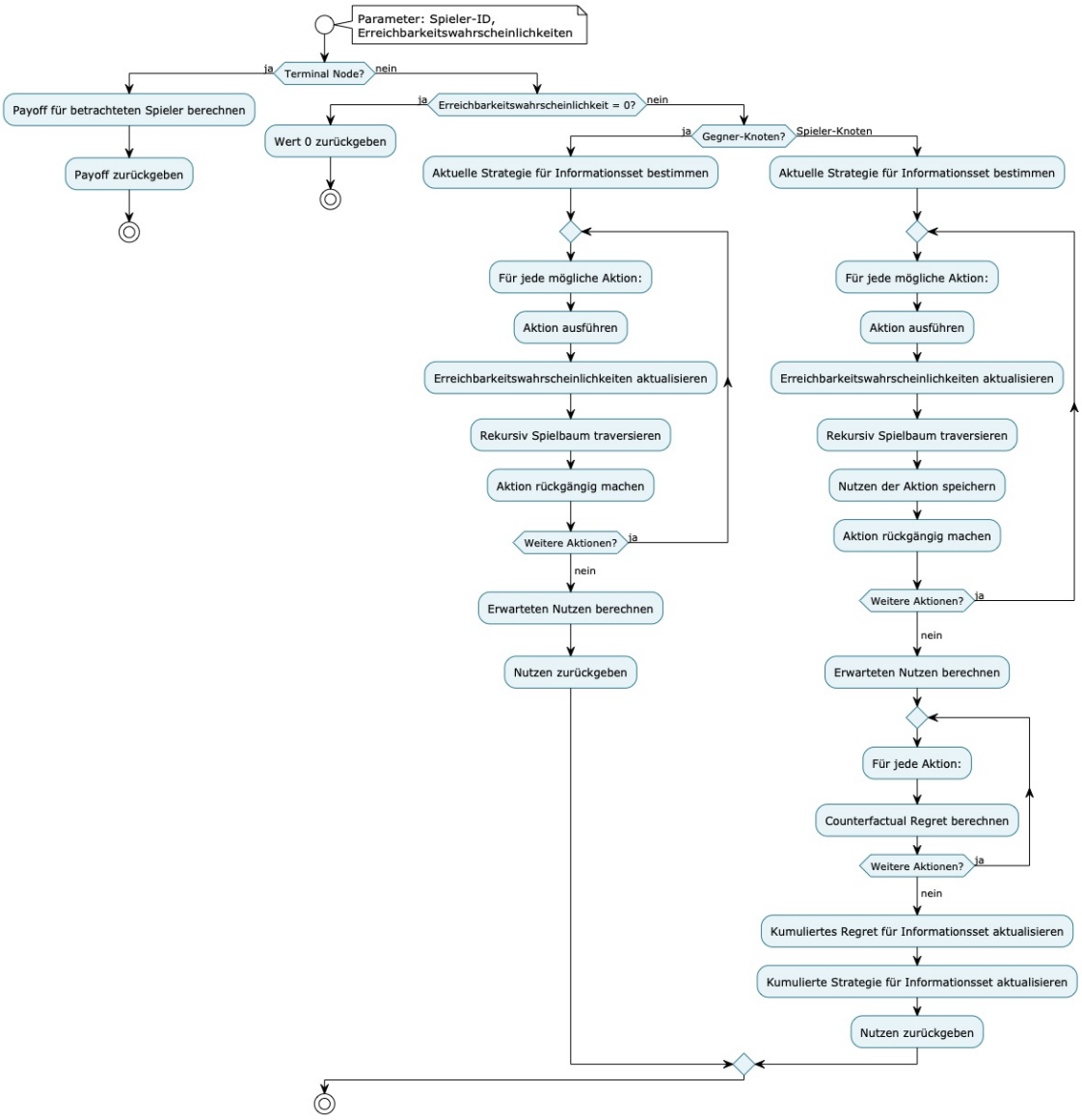

Abbildung 4.4: Aktivitätsdiagramm der Spielbaum-Traversierung der dynamischen Implementierung

4.2.2 Statischer Spielbaum

Abbildung 4.5 zeigt das Klassendiagramm der CFRSolverWithTree Klasse.

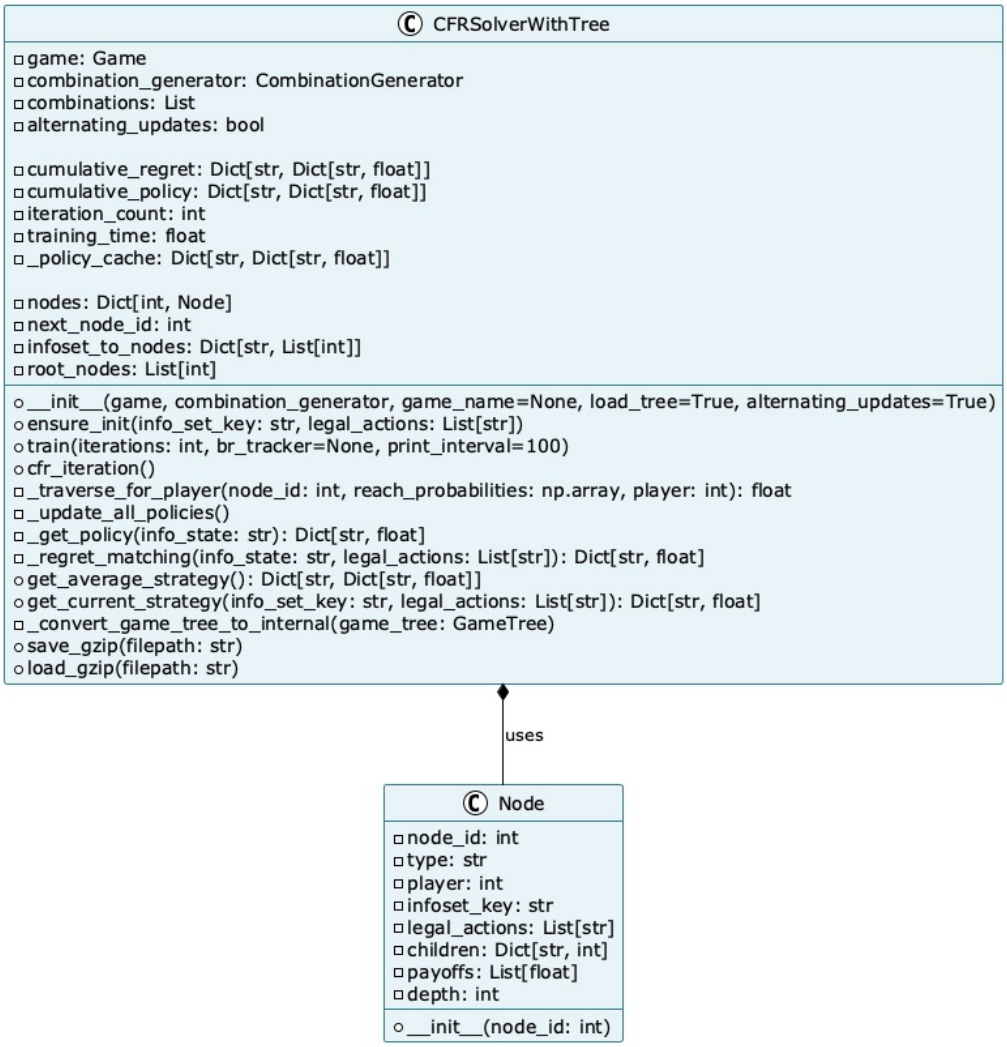

Abbildung 4.5: Klassendiagramm der Tree-basierten CFR Solver Architektur

`CFRSolverWithTree` ist die Basisklasse für alle Implementierungen, die einen statischen Spielbaum nutzen. Im Gegensatz zur dynamischen Version wird der Spielbaum einmal vor dem Training aufgebaut und als `GameTree` Objekt im Speicher gehalten. Dieses Objekt beinhaltet ein Dictionary von `Node` Objekten (`nodes = {node_id: Node}`), ein Mapping von Information Sets zu Node-IDs (`infoset_to_nodes`) und eine Liste von Root-Node-IDs (`root_nodes`). Der Tree wird als gzip-komprimiertes Pickle-Format auf der Festplatte gespeichert und kann bei späteren Trainingsläufen geladen werden, um den initialen Aufbau zu vermeiden.

Auf der folgenden Seite ist ein UML Aktivitätsdiagramm zu sehen, welches den Aufbau des Spielbaums beschreibt.

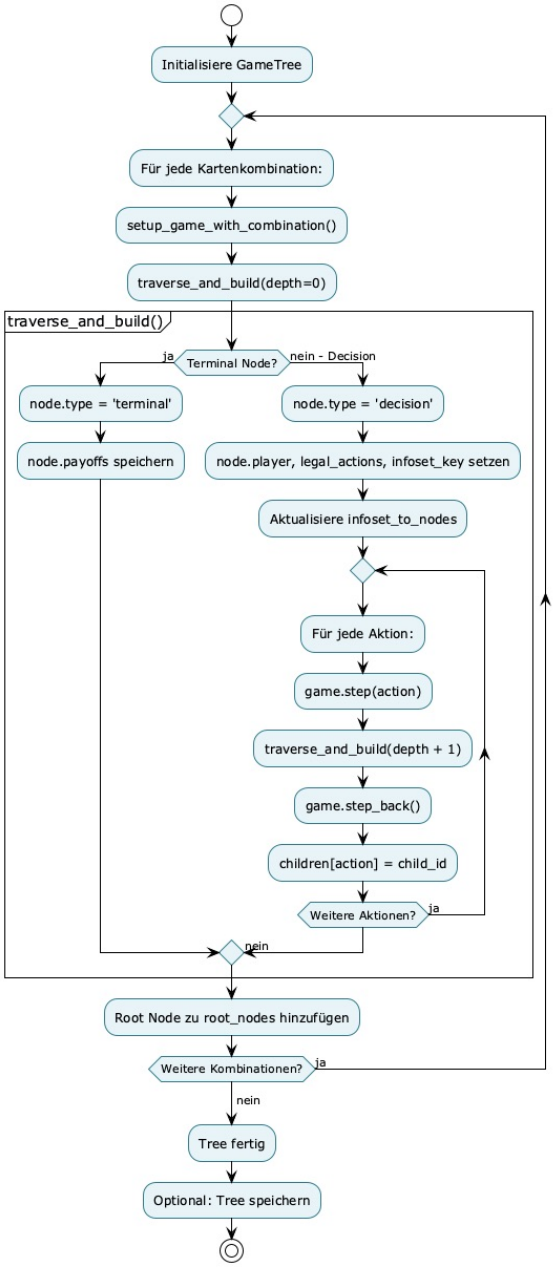

Abbildung 4.6: Aktivitätsdiagramm des Game Tree Aufbaus

Für jede Kartenkombination wird ein Subtree aufgebaut. Die Funktion `traverse_and_build()` erstellt für jeden Spielzustand einen Node, prüft ob es sich um einen Terminal Node handelt, und erstellt für jede legale Aktion rekursiv Child Nodes. Dabei werden `game.step()` und `game.step_back()` verwendet, um den Spielzustand zu verändern. Nach dem Aufbau wird der Tree als gzip-komprimiertes Pickle-Format gespeichert. Der Trainingsablauf entspricht im Wesentlichen der dynamischen Version: Für jede Kartenkombination wird der Spielbaum traversiert, wobei alternierende oder simultane Updates durchgeführt werden können. Der entscheidende Unterschied liegt in der Traversierung: Statt bei jeder Iteration `game.step()` und `game.step_back()` aufzurufen, navigiert die Traversierungsfunktion direkt durch die im Speicher vorhandene Tree-Struktur mittels Node-IDs. Die Logik der Traversierung (Gegner-Knoten vs. Spieler-Knoten, Regret-Updates) bleibt identisch zur dynamischen Version. Dieser Ansatz vermeidet wiederholte Spielzustands-

Funktionsaufrufe und ist daher deutlich schneller, benötigt jedoch mehr Arbeitsspeicher, da der gesamte Tree im Speicher gehalten werden muss.

4.3 Evaluationsmethoden

Zur Bewertung der trainierten Strategien wird der Accelerated Best Response Agent verwendet, welcher die Exploitability einer Strategie berechnet.

4.3.1 Accelerated Best Response

Der Accelerated Best Response Agent berechnet die Exploitability einer Strategie durch Traversierung des Public State Tree statt des Information Set Tree. Die theoretischen Grundlagen dieser Methode wurden bereits in Kapitel 2 beschrieben. In diesem Abschnitt wird die Architektur und der Ablauf der Implementierung beschrieben.

Der Public State Tree wird einmal vorher durch `build_public_state_tree_v2.py` erstellt und enthält Public States, Information Sets pro Spieler und Terminal Nodes. Der Tree wird als gzip-komprimiertes Pickle-Format gespeichert und kann bei Bedarf geladen werden.

Abbildung 4.7 zeigt den Gesamtablauf der Best Response Berechnung.

Die Berechnung beginnt mit dem Laden des Public State Trees und der durchschnittlichen Strategie. Für jedes Information Set des betrachteten Spielers werden die gültigen Gegner-Information Sets bestimmt (ausschließlich solche mit unterschiedlichen Karten). Die Reach Probabilities für den Gegner werden berechnet und anschließend wird der Public State Tree rekursiv traversiert. Der Gesamtnutzen wird als gewichtete Summe über alle Information Sets berechnet und als Exploitability zurückgegeben.

Abbildung 4.8 zeigt den Ablauf der rekursiven Traversierung des Public State Trees im Detail.

Die Traversierung unterscheidet zwischen drei Node-Typen: Handelt es sich um einen **Terminal Node**, wird die Hand Strength für alle Information Sets berechnet (mit Caching zur Performance-Optimierung) und die Payoffs werden als gewichtete Summe über die Reach Probabilities des Gegners berechnet.

Bei einem **Chance Node** wird für jedes Chance-Event der neue Public State bestimmt und rekursiv weiter traversiert. Der erwartete Nutzen wird als gewichtete Summe über alle Chance-Events berechnet.

Bei einem **Choice Node** wird unterschieden, ob es sich um einen Knoten des betrachteten Spielers oder des Gegners handelt. Handelt es sich um einen Knoten des **betrachteten Spielers**, wird für jede mögliche Aktion rekursiv traversiert und der Nutzen gespeichert. Für jedes Information Set wird anschließend der Best Response berechnet, indem das Maximum über alle Aktionen gewählt wird.

Handelt es sich um einen **Gegner-Knoten**, wird die Strategie des Gegners für die entsprechenden Information Sets abgerufen. Für jede mögliche Aktion werden die Reach Probabilities aktualisiert und rekursiv weiter traversiert. Der erwartete Nutzen wird als gewichtete Summe über alle Aktionen berechnet, wobei die Gewichte die Strategie-Wahrscheinlichkeiten des Gegners sind.

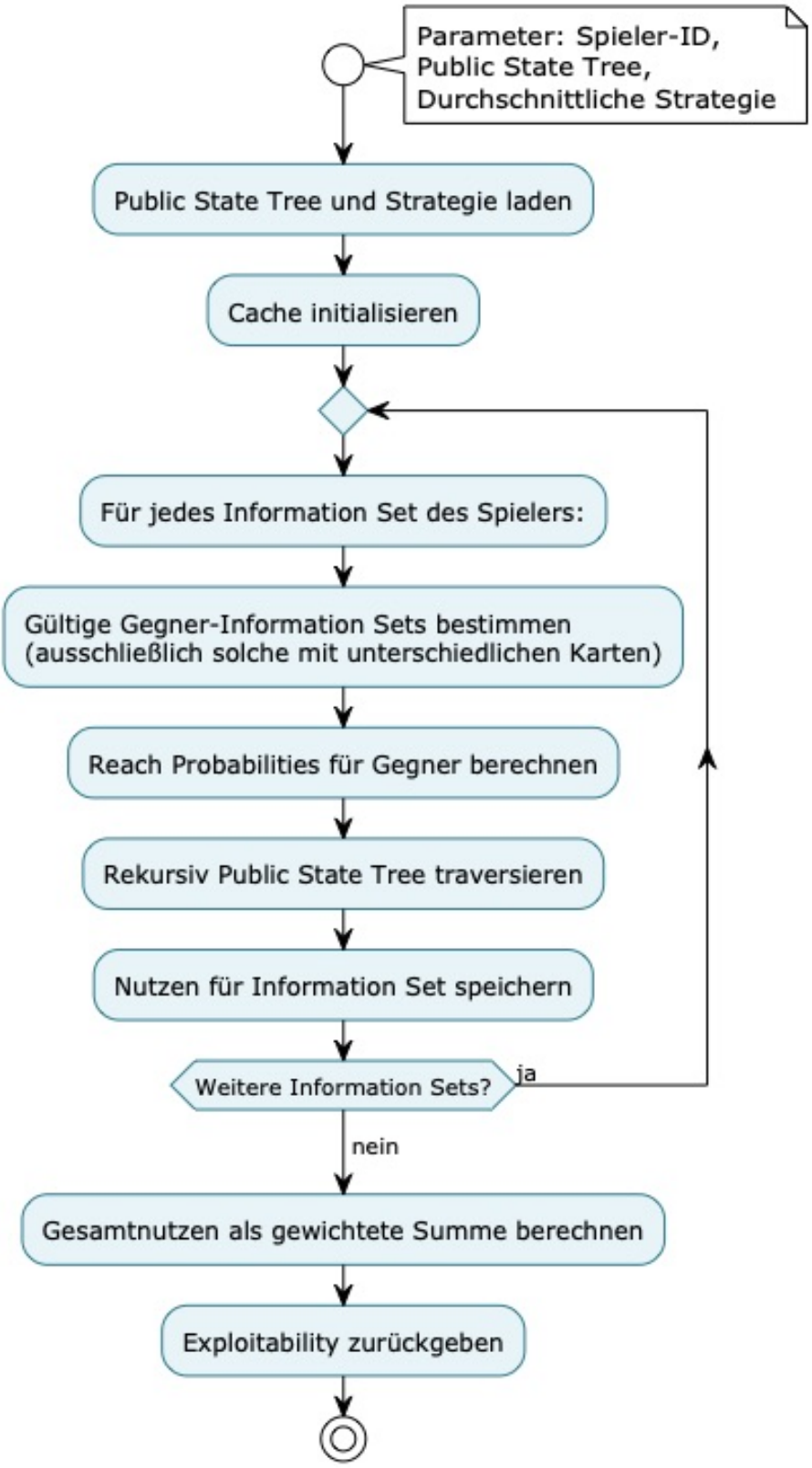

Abbildung 4.7: Aktivitätsdiagramm der Best Response Berechnung

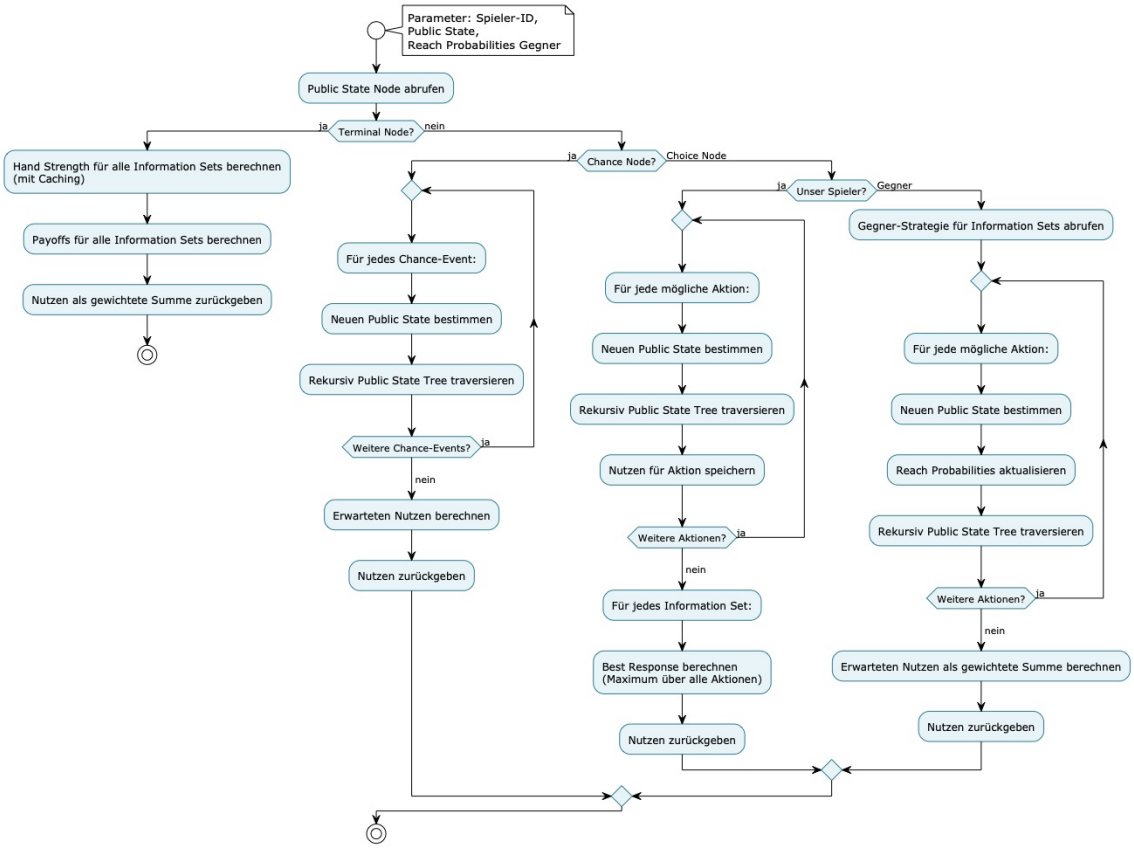

Abbildung 4.8: Aktivitätsdiagramm der Public State Tree Traversierung

Die Terminal-Node-Evaluierung nutzt Hand-Strength-Sortierung für O(n) statt O(n^2) Komplexität, und Hand Strength Evaluierungen werden gecacht, um wiederholte Berechnungen zu vermeiden. Während des Trainings kann der `BestResponseEvaluator` optional verwendet werden, um in regelmäßigen Abständen Best Response Werte zu berechnen und zu speichern, ohne den Trainingsprozess zu unterbrechen.

Kapitel 5

Implementierung

5.1 Entwicklungsvorgehen

Dieses Kapitel beschreibt meinen praktischen Entwicklungsweg von der ersten lauffähigen Implementierung bis zu dem Stand, der in Kapitel 6 evaluiert wird. Ich wiederhole dabei bewusst keine theoretischen Definitionen aus Kapitel 2 und keine Architekturdiagramme aus Kapitel 4. Stattdessen erkläre ich, welche Entscheidungen ich im Projektverlauf getroffen habe und warum diese Entscheidungen notwendig wurden. Der rote Faden des Projekts ist, dass Korrektheit in der Praxis erst mit einer belastbaren Evaluation sichtbar wird.

5.1.1 Iteratives Vorgehen statt „Big Design Up Front“

Ich habe das System schrittweise aufgebaut und dabei jeweils die nächste Engstelle adressiert, die sich im praktischen Betrieb gezeigt hat. Dabei war mein Ziel nicht, sofort eine optimale oder elegante Endarchitektur zu entwerfen, sondern schnell eine durchgängige Pipeline aus Training und Auswertung zu erhalten. Rückblickend war dieses Vorgehen sinnvoll, weil viele Probleme erst dann sichtbar wurden, als ein konkretes Evaluationssignal verfügbar war. Gleichzeitig führte das iterative Vorgehen dazu, dass frühe Architekturentscheidungen später sehr teuer zu ändern waren.

5.1.2 Phase 1: Grundgerüst und erste Solver

Am Anfang habe ich die Game Environment implementiert, sodass verschiedene Pokervarianten mit derselben Grundstruktur simuliert werden können. Auf dieser Basis habe ich zunächst einen CFR-Solver implementiert, der den Spielbaum dynamisch über `step()` und `step_back()` traversiert. Diese Version war für kleine Spiele schnell genug, um Korrektheitsprobleme prinzipiell untersuchen zu können. Anschließend habe ich Sampling-Varianten (z.B. External Sampling und Outcome Sampling) ergänzt, weil diese in der Literatur als skalierende Alternativen zu vollständigen Traversierungen beschrieben werden.

5.1.3 Phase 2: Evaluation als Diagnosetool (Public State Tree und Best Response)

Relativ früh war klar, dass eine aussagekräftige Evaluation zentral ist, weil Self-Play oder reine Trainingsmetriken die tatsächliche Spielstärke nur unzuverlässig abbilden. Ich habe daher einen Best-Response-basierten Ansatz implementiert, der die Exploitability einer gemittelten Strategie berechnet. Die technische Grundlage dafür ist ein Public State Tree, der öffentliche Zustände gruppiert und Wiederverwendung von Berechnungen ermöglicht. Der Aufbau dieses Public State Trees und die dazugehörige Best-Response-Berechnung haben im Projekt deutlich mehr Zeit beansprucht als erwartet.

5.1.4 Phase 3: Das zentrale Architekturproblem mit Chance Nodes

Im Verlauf der Implementierung des Public State Trees habe ich festgestellt, dass meine Game Environment öffentliche Karten implizit und automatisch am Rundenende dealt. Für den Public State Tree und die Best-Response-Berechnung sind jedoch explizite public chance nodes entscheidend, weil Chance-Ereignisse sonst nicht sauber modelliert und aggregiert werden können. Aus diesem Grund musste ich umfangreichen Workaround-Code schreiben, um die fehlenden expliziten Chance Nodes beim Replay der Public History zu kompensieren. Diese Workarounds haben die Evaluation schließlich funktionsfähig gemacht, aber gleichzeitig die Möglichkeit eingeschränkt, das System später „sauber“ umzubauen.

5.1.5 Phase 4: Späte Fehlererkennung und Konsequenzen

Nachdem der Best Response Agent zuverlässig lief, wurde sichtbar, dass die Solver nicht in allen Fällen wie erwartet konvergieren. Insbesondere zeigte die Exploitability, dass Fehler in der Implementierung nicht durch „optisch plausible“ Strategien im Self-Play verdeckt werden dürfen. Zusätzlich wurde deutlich, dass die Sampling-Solver in der vorliegenden Architektur nicht zuverlässig funktionieren, obwohl ihre Kurven teilweise zunächst „in die richtige Richtung“ gehen. Ein wesentlicher Grund dafür ist, dass die Architektur Kartendeals nicht als einzelne Chance-Events sampelt, sondern für Kombinationen separate Subtrees aufbaut. Damit wurde im Projektverlauf eine klare Priorität gesetzt, nämlich funktionierende CFR-basierte Volltraversierung als Referenz zu stabilisieren und Sampling nicht weiter zu forcieren.

5.1.6 Phase 5: Wechsel zur Tree-basierten Implementierung

Nachdem die Evaluation funktionsfähig war und ich belastbare Laufzeitmessungen hatte, ist mir aufgefallen, dass die wiederholten Zustandsübergänge in der Game Environment einen relevanten Laufzeitanteil verursachen. Zusätzlich wurde mir durch externes Feedback klar, dass das wiederholte „Neu-Durchlaufen“ des Baums pro Iteration vermeidbar sein müsste. Ich habe deshalb eine zweite Solver-Familie implementiert, die vor

dem Training einmal einen expliziten *Game Tree* baut und diesen anschließend direkt traversiert. Das Ziel dieses Schritts war eine bessere Laufzeit pro Iteration durch weniger Overhead in den Zustandsfunktionen. Dabei wurde aber ebenfalls sichtbar, dass ein vorgebauter Baum zwar schnell sein kann, jedoch mit zunehmender Spielgröße sehr schnell an Speichergrenzen stößt.

5.1.7 Phase 6: Pragmatische Entscheidungen und Erweiterungen

Als klar wurde, dass ein fundamentaler Fix der Chance-Node-Problematik ein Redesign der Game Environment und gleichzeitig ein Rewrite der Workarounds im Public State Tree erfordern würde, habe ich die Sampling-Solver aus dem Kernumfang der Arbeit herausgenommen. Um dennoch drei sinnvolle Vergleichsvarianten in der Evaluation zu haben, habe ich als dritte Methode Discounted CFR (DCFR) ergänzt, weil es strukturell nahe an CFR/CFR+ liegt und sich ohne kompletten Umbau integrieren ließ. In späteren Schritten habe ich außerdem Engineering-Arbeit investiert, um die Evaluation reproduzierbar zu machen, etwa durch eindeutige Output-Pfade für Ablationen und durch Batch-Skripte für Training, Plotting und Profiling.

5.1.8 Einordnung für die folgenden Abschnitte

Der restliche Kapitelteil beschreibt anschließend die wichtigsten Probleme und deren Ursachen sowie die daraus abgeleiteten Lessons Learned. Die konkreten Messergebnisse dieser Entscheidungen werden erst in Kapitel 6 anhand der Exploitability-Kurven und Laufzeitplots diskutiert.

5.2 Probleme und Herausforderungen

In diesem Abschnitt beschreibe ich die wichtigsten technischen Schwierigkeiten, die im Verlauf der Implementierung aufgetreten sind. Die Darstellung ist bewusst problemorientiert, da die Architektur und die Algorithmik bereits in Kapitel 4 bzw. Kapitel 2 erläutert wurden.

5.2.1 Fehlende explizite Chance Nodes für öffentliche Karten

Ein zentrales Problem entstand dadurch, dass öffentliche Karten in der Game Environment automatisch am Ende einer Setzrunde ausgeteilt werden. Dadurch existieren im Spielbaum keine expliziten public chance nodes, die den Kartendeal als eigene Zustandsübergänge repräsentieren. Für viele Komponenten ist diese Unterscheidung jedoch wichtig, weil Chance-Ereignisse sonst nicht sauber modelliert und wiederverwendet werden können.

5.2.2 Konsequenzen für Public State Tree und Best Response

Beim Aufbau des Public State Trees zeigte sich, dass der Best-Response-Ansatz explizite Chance-Knoten erwartet, um Zustände korrekt zu gruppieren und Reach-Probabilities konsistent zu berechnen. Um das in der bestehenden Game Environment dennoch abbilden zu können, war zusätzlicher Code notwendig, der den öffentlichen Verlauf rekonstruiert und die öffentlichen Karten an geeigneten Stellen „nachträglich“ behandelt. Diese Lösung machte die Evaluation funktionsfähig, erhöhte aber die Komplexität und erschwerte spätere strukturelle Änderungen.

5.2.3 Sampling-Solver: Ziel nicht erreicht

Zusätzlich habe ich Sampling-basierte Solver implementiert, um die in der Literatur beschriebenen Skalierungsvorteile praktisch zu nutzen. In meinem Implementierungsstand habe ich diese Solver jedoch nicht zuverlässig zum gewünschten Konvergenzverhalten gebracht. Rückblickend ist es plausibel, dass dies prinzipiell lösbar ist, jedoch eine wesentlich sorgfältigere Behandlung der Chance-Ereignisse und des Deal-Samplings erfordert hätte. Da diese Arbeiten den zeitlichen Rahmen gesprengt hätten, wurden die Sampling-Solver im weiteren Projektverlauf nicht mehr als Kernvergleichsbaseline betrachtet.

5.2.4 Skalierungsgrenzen am Beispiel großer Hold’em-Varianten

Mit steigender Spielkomplexität verschieben sich die praktischen Engpässe von „funktioniert grundsätzlich“ hin zu „passt noch in Speicher und Laufzeitbudget“. Für Heads-Up Limit Hold’em war in meinem Projektstand insbesondere der Zustandsraum so groß, dass Tree-basierte Ansätze in der Praxis nicht mehr einsetzbar waren. Für solche Spiele fallen damit alle Solver-Varianten weg, die einen vollständigen Game Tree vorab materialisieren müssen. Gleichzeitig bedeutet das nicht, dass Training in größeren Spielen grundsätzlich unmöglich ist, sondern dass die aktuelle Implementierung ohne weitere Maßnahmen sehr langsam wird. Als grobe Größenordnung habe ich für Rhode Island Hold’em geschätzt, dass eine einzelne Iteration in meinem aktuellen Implementierungsstand etwa 15 Stunden dauern würde. Um große Varianten wie Limit Hold’em praktisch bearbeiten zu können, wären daher mehrere weitere Schritte notwendig. Erstens wäre eine stärkere Optimierung der Implementation erforderlich, da in Python ein erheblicher Overhead entsteht und sich durch effizientere Datenstrukturen oder native Komponenten vermutlich viel Laufzeit einsparen lässt. Zweitens ist Abstraktion notwendig, um den Zustandsraum so zu reduzieren, dass Training und Evaluation in realistischer Zeit möglich werden. Drittens wäre Parallelisierung sinnvoll, um unabhängige Teilprobleme (z.B. Deals oder Teilbäume) auf mehrere Kerne zu verteilen. Viertens sind zusätzliche Optimierungsverfahren wie Pruning oder verwandte Beschleunigungen hilfreich, um unnötige Traversierungen zu vermeiden und Rechenzeit gezielt dort zu investieren, wo sie das Ergebnis beeinflusst.

5.2.5 Nicht umgesetzte Abstraktion: Hand-strength-squared

Eine zusätzliche Abstraktionsidee war eine Reduktion über Hand-strength-squared. Diese Erweiterung wurde im Projektverlauf geplant, konnte jedoch aus Zeitgründen nicht mehr umgesetzt werden.

5.3 Erkenntnisse und Reflexion

In diesem Abschnitt fasse ich Erkenntnisse zusammen, die sich direkt aus dem Implementierungsprozess ergeben haben. Diese Erkenntnisse sind bewusst von den experimentellen Ergebnissen aus Kapitel 6 getrennt. Im Fazit in Kapitel 7 werden diese Punkte erneut aufgegriffen, dort jedoch stärker im Kontext der Zielerreichung und der Gesamtaussagen der Arbeit.

5.3.1 Was sich im Projektverlauf als wichtig erwiesen hat

Im Projektverlauf hat sich gezeigt, dass eine funktionsfähige Evaluation der wichtigste Hebel zur Sicherstellung von Korrektheit ist. Ohne ein verlässliches Exploitability-Signal können Implementierungsfehler lange unentdeckt bleiben. Darüber hinaus hat sich gezeigt, dass Architekturentscheidungen, die zunächst wie reine Implementierungsdetails wirken, spätere Arbeiten stark begrenzen können.

5.3.2 Lessons Learned aus der Implementierung

Best Response als Diagnosetool war entscheidend. Ich konnte erst nach Fertigstellung des Best Response Agents zuverlässig erkennen, ob ein Solver tatsächlich in Richtung Nash-Gleichgewicht konvergiert. **Fehler wurden dadurch spät sichtbar.** Der lange Aufwand für Public State Tree und Chance-Node-Workarounds hat dazu geführt, dass Korrektheitsprobleme in den Solvern erst spät im Projektverlauf auffielen. **Workaround-Schulden wachsen schnell.** Je mehr Sonderfalllogik im Public State Tree gesammelt wurde, desto teurer wurde es, das zugrundeliegende Architekturproblem sauber zu beheben. **„Dynamisch ist dumm“ war zu kurz gedacht.** Der Treebasierte Ansatz kann Laufzeit sparen, führt aber bei großen Spielen schnell zu Speicherproblemen und ist daher nicht universell einsetzbar. **Pragmatische Entscheidungen sind manchmal notwendig.** Das Aufgeben der Sampling-Solver und das Ergänzen von DCFR war eine pragmatische Reaktion, um den Vergleich in Kapitel 6 weiterhin sinnvoll durchführen zu können.

5.3.3 Was ich beim nächsten Mal anders machen würde

Ich würde die Evaluationspipeline früher implementieren, um Korrektheitsprobleme der Solver früh zu erkennen. Ich würde die Behandlung von Chance-Ereignissen in der Game

Environment so entwerfen, dass öffentliche Kartendeals explizit modelliert werden können. Ich würde Skalierungsfragen früher prototypisch prüfen, um Speicher- und Laufzeitgrenzen nicht erst bei sehr großen Varianten zu entdecken. Ich würde meine Literaturrecherche am Anfang breiter anlegen und mich nicht so früh auf wenige Standardvarianten wie CFR, CFR+ und MCCFR versteifen. Ich würde gezielt nach Überblicksquellen suchen, die die wichtigsten Optimierungen und Varianten systematisch einordnen, bevor ich mich auf drei konkrete Algorithmen festlege. Dadurch hätte ich früher gesehen, dass es neben CFR+ weitere Ansätze mit anderen Gewichtungs- und Update-Schemata gibt, die für Vergleiche und Implementierungsentscheidungen relevant sind. Als Einstieg in diese „Landkarte“ wäre ein gut kuratierter Kursüberblick wie z. B. *Computational Game Solving* hilfreich gewesen.[1] Die Folien sind didaktisch deutlich schneller zugänglich als viele Originalpapers und hätten mir zu Projektbeginn Orientierung, Begriffe und Prioritäten klarer vermittelt.

[1] `https://www.cs.cmu.edu/~sandholm/cs15-888F25/`

Kapitel 6

Evaluation

6.1 Ergebnisse

Kuhn Poker Ergebnisse

Leduc Hold'em Ergebnisse

Eigene Version

Rhode Island Hold'em Ergebnisse

6.2 Vergleich der Algorithmen

CFR vs. CFR+

CFR vs. MCCFR

Konvergenzplots

Abstraktionsmethoden, Multithread

6.3 Vergleich mit Literaturwerten und anderen Modellen

Kuhn 1951 (analytische Lösung)

Zinkevich 2007 (CFR)

Lanctot et al. 2009 (MCCFR)

Tammelin et al. 2015 (CFR+)

OpenSpiel/RL Card

Random Leute in Foren

Kapitel 7

Fazit

7.1 Zielerreichung und Einordnung

In diesem Kapitel fasse ich die Ergebnisse der Arbeit zusammen und ordne sie im Hinblick auf die Zielsetzung aus Kapitel 1 ein. Ich führe dabei keine neuen Experimente ein, sondern beziehe mich auf die in Kapitel 6 dargestellten Kurven und Messwerte. Der Schwerpunkt der Arbeit liegt auf der Implementierung einer durchgängigen Pipeline aus Training und Evaluation für CFR-basierte Verfahren in Pokerspielen.

7.1.1 Kurzüberblick

Ich habe mehrere Pokervarianten in einer einheitlichen Game-Environment-Struktur implementiert. Darauf aufbauend habe ich mehrere CFR-Varianten implementiert und in verschiedenen Implementierungsformen ausgeführt. Zur Bewertung der Strategien habe ich Exploitability über einen Best-Response-Ansatz auf Basis eines Public State Trees berechnet. Zusätzlich habe ich den Trainingsprozess so automatisiert, dass Trainingsläufe und Auswertungsartefakte reproduzierbar erzeugt werden können.

7.1.2 Zielerreichung (Bezug zu Kapitel 1)

Im Folgenden bewerte ich die Zielerreichung in kompakter Form. Die detaillierten Ergebnisse und Kurven sind in Kapitel 6 dokumentiert.

Erreichte Ziele

- **Anwendung von CFR auf zunehmend komplexe Pokervarianten bis Twelve Card Poker.** Ich habe die Pipeline so umgesetzt, dass Strategien für kleine und mittlere Spiele berechnet werden können und Twelve Card Poker als Zielspiel trainiert und evaluiert werden kann.
- **Implementierung mehrerer CFR-Varianten für einen sinnvollen Vergleich.** Neben Vanilla CFR habe ich weitere Varianten implementiert, um Konvergenz-

und Laufzeitunterschiede im Rahmen derselben Spielumgebung vergleichen zu können.

- **Evaluation über Exploitability.** Ich habe eine Exploitability-basierte Evaluation über Best Response implementiert und damit die Spielstärke gemittelter Strategien messbar gemacht.
- **Vergleich von Implementierungsvarianten.** Ich habe dynamische Traversierung und Tree-basierte Traversierung gegenübergestellt und die jeweiligen Tradeoffs diskutiert.
- **Reproduzierbarkeit der Experimente.** Durch Skripte und feste Output-Pfade lassen sich Training und Auswertung in konsistenter Form wiederholen.

Nicht erreichte Ziele

- **Umsetzung von Sampling-Solvern als geplante Optimierungsverfahren.** Im Exposé war vorgesehen, neben CFR+ auch Sampling-basierte Verfahren (MCCFR) praktisch zu vergleichen. Die dafür implementierten Solver konnten im Projektstand jedoch nicht so stabilisiert werden, dass sie als belastbarer Vergleich in die Evaluation eingehen.

7.1.3 Rückbindung an Hypothesen und Fragestellungen

Sofern in Kapitel 1 Hypothesen oder Forschungsfragen formuliert werden, können diese an dieser Stelle abschließend eingeordnet werden. Für Hypothesen, die sich direkt auf Spielgröße, Iterationslaufzeiten oder Speicherbedarf beziehen, liefern die Messartefakte aus Kapitel 6 eine belastbare Grundlage. Für qualitative Hypothesen über „menschenähnliche“ Taktiken ist eine saubere Belegung schwieriger, wenn ausschließlich Exploitability- und Laufzeitkurven betrachtet werden. In diesem Fall ist es sinnvoll, solche Aussagen als offene Frage zu markieren und in den Ausblick zu verschieben.

7.2 Limitierungen

In diesem Abschnitt formuliere ich die wichtigsten Limitierungen der Arbeit und grenze die Aussagekraft der Ergebnisse ein. Die Punkte sind nicht als Entschuldigung zu verstehen, sondern als klare Beschreibung der Bedingungen, unter denen die gezeigten Resultate gelten.

7.2.1 Iterationslaufzeit als primäre Skalierungsgrenze

Die wichtigste praktische Grenze ist die Laufzeit pro Iteration bei großen Spielen. Mit zunehmender Spielgröße wird bereits eine vollständige Traversierung in Python so teuer, dass Training in vertretbarer Zeit nicht mehr möglich ist. Als Größenordnung ergibt sich für Rhode Island Hold’em in meinem Projektstand eine Iterationslaufzeit in der Größenordnung von etwa 15 Stunden.

7.2.2 Speicherbedarf bei Tree-basierten Varianten

Tree-basierte Solver reduzieren Overhead durch Zustandsübergänge, benötigen aber Speicher für den vollständigen Game Tree. Bei großen Varianten kann dadurch nicht nur Training, sondern bereits das Bauen und Halten der Baumstruktur praktisch begrenzt werden.

7.2.3 Evaluation hängt von PST und Best Response ab

Exploitability ist nur so zuverlässig wie die Implementierung der Best-Response-Berechnung und des Public State Trees. Fehler oder Inkonsistenzen in dieser Pipeline würden sich direkt auf die gemessene Exploitability auswirken. Als Plausibilitätscheck kann für Leduc zusätzlich der erwartete Spielwert betrachtet werden, bei dem sich ein konsistentes Zero-Sum-Verhalten als $+83\,\mathrm{mbb/g}$ bzw. $-83\,\mathrm{mbb/g}$ zeigt.

7.2.4 Messdichte und Zeitmessungen

Für größere Spiele sind Best-Response-Messpunkte nur sparsam möglich, weil die Evaluation zusätzliche Laufzeit verursacht. Zeitplots sind grundsätzlich als Orientierung zu interpretieren, weil Hintergrundlast und Hardwareunterschiede die absolute Wall-Clock-Time beeinflussen können. Die betrachteten Volltraversierungs-Solver sind in meinem Setup weitgehend deterministisch, weshalb keine Varianzabschätzung über Seeds im klassischen Sinn erfolgt.

7.3 Future Work

In diesem Abschnitt skizziere ich konkrete nächste Schritte, die sich aus den Ergebnissen und Limitierungen der Arbeit ergeben. Ich trenne dabei zwischen Implementierungsarbeit zur Skalierung der bestehenden Pipeline und algorithmischen Erweiterungen, die den Zustandsraum und die Konvergenz beeinflussen.

7.3.1 Implementierung und Engineering

Ein zentraler nächster Schritt ist die Reduktion der Iterationslaufzeit für große Spiele. Dazu gehören effizientere Datenstrukturen, weniger Python-Overhead und eine stärkere Parallelisierung unabhängig bearbeitbarer Teilprobleme. Zusätzlich ist eine konsequente Profiling-basierte Optimierung sinnvoll, um Hotspots in Traversierung, Handbewertung und Evaluation gezielt zu verbessern. Für Tree-basierte Varianten sind kompaktere Repräsentationen und speichereffiziente Serialisierung relevant, um Speichergrenzen später zu erreichen. Darüber hinaus ist die Weiterentwicklung der Tooling-Schicht sinnvoll, damit Experimente, Plots und Profile für Dritte einfacher reproduzierbar sind.

7.3.2 Algorithmische und wissenschaftliche Erweiterungen

Für die Skalierung auf große Hold'em-Varianten sind zusätzliche algorithmische Techniken notwendig. Ein naheliegender Ansatz sind Pruning-Methoden, die unnötige Traversierungen vermeiden und Rechenzeit stärker auf relevante Bereiche konzentrieren [Brown and Sandholm, 2015]. Wenn Sampling-basierte Verfahren erneut betrachtet werden sollen, sind effizientere MCCFR-Varianten für Spiele mit vielen Aktionen und stabilere Praxis-Implementierungen relevant [Lanctot et al., 2009, Gibson et al., 2012]. Für sehr große Spiele ist Abstraktion ein zentraler Hebel, um den Zustandsraum zu reduzieren, ohne die grundlegende Spielstruktur vollständig zu verlieren [Gilpin and Sandholm, 2007, Billings et al., 2003]. Als zusätzliche wissenschaftliche Erweiterung ist eine systematische Größenabschätzung und Messung von Spielinstanzen hilfreich, um Skalierungsgrenzen früh zu erkennen und Experimente gezielt zu planen [Johanson, 2013]. Schließlich können weiterführende Spiel-Solving-Methoden aus der Literatur als langfristige Perspektive dienen, etwa für den Übergang von Toy Games zu vollständigen Hold'em-Varianten [Bowling et al., 2015, Brown and Sandholm, 2019b].

Kapitel 8

Anhang

8.1 M-Wert

Lanctot et al. [Lanctot et al., 2009] führen zur Formulierung der Regret Bounds ein strukturabhängiges Maß der Spielkomplexität ein: den M-Wert. Dieser Wert hängt von der spezifischen Struktur des extensiven Spiels ab und ermöglicht eine präzisere Abschätzung der Konvergenzrate als die einfache Anzahl der Information Sets.

Sei $\vec{a}_i$ eine Teilsequenz einer Historie, die nur die Aktionen von Spieler i in dieser Historie enthält. Sei $\vec{A}_i$ die Menge aller solcher Aktionssequenzen von Spieler i. Für eine Aktionssequenz $\vec{a}_i$ sei $\mathcal{I}_i(\vec{a}_i)$ die Menge aller Information Sets, bei denen die Aktionssequenz von Spieler i bis zu diesem Information Set genau $\vec{a}_i$ ist. Der M-Wert für Spieler i ist dann definiert als

$$M_i = \sum_{\vec{a}_i \in \vec{A}_i} \sqrt{|\mathcal{I}_i(\vec{a}_i)|}. \tag{8.1}$$

Der M-Wert summiert also über alle möglichen Aktionssequenzen von Spieler i die Wurzel der Anzahl der Information Sets, die zu dieser Sequenz gehören. Dies erfasst, wie sich die Information Sets über die verschiedenen Aktionssequenzen verteilen.

Für den M-Wert gilt die folgende Ungleichung:

$$\sqrt{|\mathcal{I}_i|} \leq M_i \leq |\mathcal{I}_i|, \tag{8.2}$$

wobei beide Seiten dieser Schranke von bestimmten Spielen erreicht werden können ([Lanctot et al., 2009]).

Die untere Schranke $\sqrt{|\mathcal{I}_i|}$ wird erreicht, wenn alle Information Sets zu derselben Aktionssequenz gehören. Die obere Schranke $|\mathcal{I}_i|$ wird erreicht, wenn jede Aktionssequenz zu genau einem Information Set führt.

Als Beispiel dient Kuhn Poker. Beide Spieler haben jeweils sechs Information Sets: $|\mathcal{I}_0| = |\mathcal{I}_1| = 6$.

Für Spieler 0 gibt es zwei Aktionssequenzen mit Information Sets: die leere Sequenz [] mit drei Information Sets $(J, (), 0)$, $(Q, (), 0)$, $(K, (), 0)$ am Spielbeginn, sowie die Sequenz $['check']$ mit drei Information Sets $(J, ('check', 'bet'), 0)$, $(Q, ('check', 'bet'), 0)$,

$(K, ('check', 'bet'), 0)$ nach einem Check von Spieler 0 und einem Bet von Spieler 1. Somit gilt $M_0 = \sqrt{3} + \sqrt{3} = 2\sqrt{3} \approx 3.46$.

Für Spieler 1 gibt es nur eine Aktionssequenz mit Information Sets: die leere Sequenz [] mit sechs Information Sets, nämlich drei nach einem Check von Spieler 0 und drei nach einem Bet von Spieler 0. Somit gilt $M_1 = \sqrt{6} \approx 2.45$.

Die Verifikation der Schranken ergibt: $\sqrt{6} \approx 2.45 \leq M_0 = 3.46 \leq 6$ und $\sqrt{6} \approx 2.45 \leq M_1 = 2.45 \leq 6$. Für Spieler 1 wird die untere Schranke erreicht, da alle sechs Information Sets zur selben Aktionssequenz gehören. Für Spieler 0 liegt M_0 zwischen den Schranken, da die Information Sets auf zwei verschiedene Aktionssequenzen verteilt sind.

Der M-Wert wird in den Regret Bounds für Vanilla CFR und MCCFR verwendet.

Literaturverzeichnis

[Billings et al., 2003] Billings, D., Burch, N., Davidson, A., Holte, R., Schaeffer, J., Schauenberg, T., and Szafron, D. (2003). Approximating game-theoretic optimal strategies for full-scale poker. In *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI 2003)*. PDF: `https://poker.cs.ualberta.ca/publications/IJCAI03.pdf` (Zugriff: 19.01.2026).

[Bowling et al., 2015] Bowling, M., Burch, N., Johanson, M., and Tammelin, O. (2015). Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149. doi: `https://doi.org/10.1126/science.1259433`.

[Brown and Sandholm, 2015] Brown, N. and Sandholm, T. (2015). Regret-based pruning in extensive-form games. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, pages 1972–1980. PDF: `https://www.cs.cmu.edu/~sandholm/regret-basedPruning.nips15.withAppendix.pdf` (Zugriff: 19.01.2026).

[Brown and Sandholm, 2017] Brown, N. and Sandholm, T. (2017). Libratus: The superhuman ai for no-limit poker (demonstration). In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-17)*, pages 5226–5228. PDF: `https://www.ijcai.org/proceedings/2017/0772.pdf` (Zugriff: 20.01.2026).

[Brown and Sandholm, 2018] Brown, N. and Sandholm, T. (2018). Superhuman ai for heads-up no-limit poker: Libratus beats top professionals. *Science*, 359(6374):418–424. doi: `https://doi.org/10.1126/science.aao1733`.

[Brown and Sandholm, 2019a] Brown, N. and Sandholm, T. (2019a). Solving imperfect-information games via discounted regret minimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1829–1836. arXiv:1809.04040, `https://arxiv.org/pdf/1809.04040` (Zugriff: 19.01.2026).

[Brown and Sandholm, 2019b] Brown, N. and Sandholm, T. (2019b). Superhuman ai for multiplayer poker. *Science*, 365(6456):885–890. doi: `https://doi.org/10.1126/science.aay2400`.

[Gibson et al., 2012] Gibson, R. G., Burch, N., Lanctot, M., and Szafron, D. (2012). Efficient monte carlo counterfactual regret minimization in games with many player actions. In *Advances in Neural Information Processing Systems 26 (NIPS 2012)*, pages 1889–1897. PDF: `https://papers.nips.cc/paper/2012/file/3df1d4b96d8976ff5986393e8767f5b2-Paper.pdf` (Zugriff: 19.01.2026).

[Gilpin and Sandholm, 2007] Gilpin, A. and Sandholm, T. (2007). Lossless abstraction for imperfect information games. In *Proceedings of the National Conference on Artificial Intelligence*, volume 22, page 343. doi: https://doi.org/10.1145/1284320.1284324.

[Herbster and Warmuth, 1998] Herbster, M. and Warmuth, M. K. (1998). Tracking the best expert. *Machine Learning*, 32(2):151–178. doi: https://doi.org/10.1023/A:1007424614876.

[Johanson et al., 2011] Johanson, M., Waugh, K., Bowling, M., and Zinkevich, M. (2011). Accelerating best response calculation in large extensive games. In *IJCAI*. https://www.ijcai.org/Abstract/11/054.

[Johanson, 2013] Johanson, M. B. (2013). Measuring the size of large no-limit poker games. Technical report tr-13-01, Department of Computing Science, University of Alberta. arXiv:1302.7008, https://arxiv.org/pdf/1302.7008 (Zugriff: 19.01.2026).

[Kuhn, 1951] Kuhn, H. W. (1951). Simplified two-person poker. *Contributions to the Theory of Games*, 1:97–103. https://books.google.com/books/about/Contributions_to_the_Theory_of_Games.html?id=TpbbVU4tA58C.

[Lanctot et al., 2009] Lanctot, M., Waugh, K., Zinkevich, M., and Bowling, M. (2009). Monte carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems*, pages 1078–1086. https://proceedings.neurips.cc/paper/2009/hash/00411460f7c92d2124a67ea0f4cb5f85-Abstract.html.

[Osborne and Rubinstein, 1994] Osborne, M. J. and Rubinstein, A. (1994). *A Course in Game Theory*. MIT Press, Cambridge, Massachusetts. https://mitpress.mit.edu/9780262650403/a-course-in-game-theory.

[Shi and Littman, 2001] Shi, J. and Littman, M. L. (2001). Rhode island hold'em poker. In *AAAI/IAAI*, pages 602–607. https://www.cs.cmu.edu/~gilpin/gsi.html.

[Southey et al., 2005] Southey, F., Bowling, M., Larson, B., Piccione, C., Burch, N., Billings, D., and Rayner, C. (2005). Bayes' bluff: Opponent modelling in poker. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence (UAI 2005)*. https://webdocs.cs.ualberta.ca/~bowling/publications/b2hd-05uai.html.

[Tammelin et al., 2015] Tammelin, O., Burch, N., Johanson, M., and Bowling, M. (2015). Solving heads-up limit texas hold'em. In *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence (IJCAI 2015)*, pages 645–652. https://www.ijcai.org/Abstract/15/097.

[Zinkevich et al., 2007] Zinkevich, M., Johanson, M., Bowling, M., and Piccione, C. (2007). Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems*. https://papers.nips.cc/paper/3306-regret-minimization-in-games-with-incomplete-information.